

NC DHHS Grant Finder: source code

Every script, prompt and configuration file of the pipeline, in run order. Tests (71) are in `tests/` of the repository.

1. `src/pipeline.py`
2. `src/config.py`
3. `src/clean_data.py`
4. `src/deterministic.py`
5. `src/semantic_triage.py`
6. `src/deep_analysis.py`
7. `src/scoring.py`
8. `src/award_history.py`
9. `src/validation.py`
10. `src/keyword_baseline.py`
11. `src/report.py`
12. `src/submission_table.py`
13. `src/clients.py`
14. `src/llm_cache.py`
15. `src/text_utils.py`
16. `prompts/jev_triage.json`
17. `prompts/deep_analysis.txt`
18. `agency/priorities.json`

src/pipeline.py

```
"""Run the whole pipeline end to end.

    python src/pipeline.py          # real run; model calls come from data/cache/ when cached
    python src/pipeline.py --mock   # dry run with fake model answers (data/results/mock/)

Order: Layer 0 clean -> Layer 1 features -> keyword baseline -> Layer 2 Jev triage
-> Layer 3 deep analysis -> Layer 4 scoring -> top-results report -> historical-award
check (USAspending) -> validation report (if labels exist).

With every model response committed under data/cache/, a real run makes no API
calls and needs no key: the same raw data + config + prompts + cache always give
the same ranking.
"""

import argparse
import sys

import award_history
import clean_data
import config
import deep_analysis
import deterministic
import keyword_baseline
import report
import scoring
import semantic_triage
import submission_table
import validation

def main(mock: bool) -> None:
    print("=== Layer 0: clean ===")
    clean_data.main()
    print("\n=== Layer 1: deterministic features ===")
    deterministic.main()
    print("\n=== Layer 5 baseline: keyword TF-IDF ===")
    keyword_baseline.run()

    if not mock and not config.api_key():
        print("\nNote: no AI_GATEWAY_API_KEY - Layers 2-3 will use cached answers only; "
              "any grant not in data/cache/ will fail and be reported.")
    print("\n=== Layer 2: Jev triage ===")
    l2 = semantic_triage.run(mock=mock)
    if l2.empty:
        sys.exit("Layer 2 produced no results (no API key and no cache?). Add the key to .env or use --mock.")
    print("\n=== Layer 3: deep analysis ===")
    deep_analysis.run(mock=mock)
    print("\n=== Layer 4: scoring ===")
    scoring.run(mock=mock)
    print("\n=== Report ===")
    report.run(mock=mock)

    if not mock:
        print("\n=== Layer 5: historical-award check (USAspending.gov, cached) ===")
        award_history.run()
        print("\n=== Submission table ===")
        submission_table.run()

    if not mock and validation.LABELS.exists():
        print("\n=== Layer 5: validation ===")
        try:
            validation.cmd_evaluate()
        except SystemExit as e: # no labels filled in yet
            print(e)
```

```
if __name__ == "__main__":  
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)  
    ap.add_argument("--mock", action="store_true", help="fake model answers; no API key needed")  
    main(ap.parse_args()).mock)
```

src/config.py

```
"""Paths, agency configuration and model settings shared by every layer."""

import json
import os
from pathlib import Path

from dotenv import load_dotenv

ROOT = Path(__file__).resolve().parent.parent
load_dotenv(ROOT / ".env")

DATA = ROOT / "data"
RAW_GRANTS = DATA / "raw" / "grants.csv"
LAYER1_OUTPUT = DATA / "processed" / "grants_features.csv"
RESULTS = DATA / "results"
CACHE = DATA / "cache" # real model responses, committed so reruns need no API keys
MOCK_RESULTS = RESULTS / "mock" # mock-mode outputs, never mixed with real ones
MOCK_CACHE = DATA / "cache_mock"

AGENCY_CONFIG = ROOT / "agency" / "priorities.json"
PROMPTS = ROOT / "prompts"

# Models (both reached through the Vercel AI Gateway with one key)
JEV_MODEL = os.getenv("JEV_MODEL", "typesafe-ai/jev")
DEEP_MODEL = os.getenv("DEEP_MODEL", "deepseek/deepseek-v4-pro")
GATEWAY_EVALUATE_URL = "https://ai-gateway.vercel.sh/v1/evaluate"
GATEWAY_CHAT_URL = "https://ai-gateway.vercel.sh/v1/chat/completions"
REQUEST_TIMEOUT_S = 90

def api_key() -> str:
    return os.getenv("AI_GATEWAY_API_KEY", "").strip()

def load_agency() -> dict:
    with open(AGENCY_CONFIG, encoding="utf-8") as f:
        return json.load(f)

PROMPT_FIELDS = ("agency_name", "mission", "user", "strategic_priorities", "divisions")

def prompt_fields(agency: dict) -> dict:
    """The parts of the agency config that are sent to a model. Only these go into
    the prompt-version hash, so tuning thresholds or Layer 4 weights does not
    invalidate the cache of model answers."""
    return {k: agency[k] for k in PROMPT_FIELDS}

def objective_ids(agency: dict) -> set[str]:
    return {o["id"] for p in agency["strategic_priorities"] for o in p["objectives"]}
```

src/clean_data.py

```
"""Layer 0 - data cleaning and normalization. No judgment about relevance.

python src/clean_data.py

Reads data/raw/grants.csv
Writes data/processed/grants_cleaned.csv

What it does: parse dates and numbers, normalize booleans, drop exact duplicate
opportunity ids, add `grant_id` (= opportunity_id) and plain-text helper columns.

What it deliberately does NOT do:
* Text columns (title, summary, eligibility, applicant types, listings, ...) stay
  exactly as exported, raw HTML and trailing spaces included. Layers 2-3 strip HTML
  themselves when they build prompts, and unchanged text keeps their cached model
  responses valid. HTML-free versions are added as `summary_text` and `grant_text`.
* Blanks stay blank. A blank `is_cost_sharing` is unknown, not False.
* Zero award amounts are kept as 0 here; Layer 1 decides they are placeholders.
"""

import pandas as pd

import config
from text_utils import strip_html

CLEANED = config.DATA / "processed" / "grants_cleaned.csv"

DATE_COLS = ["post_date", "close_date", "archive_date", "forecasted_post_date",
             "forecasted_close_date", "forecasted_award_date", "forecasted_project_start_date"]
NUM_COLS = ["award_floor", "award_ceiling", "estimated_total_program_funding",
            "expected_number_of_awards"]
BOOL_COLS = ["is_forecast", "is_cost_sharing"]
TEXT_COLS = ["opportunity_title", "opportunity_status", "agency_name", "top_level_agency_name",
             "summary_description", "applicant_types", "applicant_eligibility_description",
             "opportunity_assistance_listings", "funding_instruments"]
_BOOL_MAP = {"true": True, "yes": True, "1": True, "false": False, "no": False, "0": False}

def to_bool(series: pd.Series) -> pd.Series:
    """True/'False' -> bool; anything else (blank, junk) -> <NA>."""
    return series.astype(str).str.strip().str.lower().map(_BOOL_MAP).astype("boolean")

def clean(raw: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    """Return the cleaned frame plus a small report of what was fixed or dropped."""
    df = raw.copy()
    report = {"rows_in": len(df)}

    # Text is kept exactly as exported (no trimming): Layers 2-3 build their prompts, and
    # their cache keys, from these strings. Only the status is trimmed, because Layer 1
    # compares it against fixed words.
    for c in TEXT_COLS:
        df[c] = df[c].fillna("").astype(str) if c in df else ""
    df["opportunity_status"] = df["opportunity_status"].str.strip()

    # Dates and numbers. Count values that were present but unparseable.
    report["unparseable"] = {}
    for c in DATE_COLS + NUM_COLS:
        if c not in df:
            df[c] = pd.NaT if c in DATE_COLS else float("nan")
            continue
        present = df[c].astype(str).str.strip() != ""
        parsed = (pd.to_datetime(df[c], errors="coerce") if c in DATE_COLS
                  else pd.to_numeric(df[c], errors="coerce"))
        bad = int((present & parsed.isna()).sum())
        if bad:
            report["unparseable"][c] = bad
```

```

df[c] = parsed

for c in BOOL_COLS:
    df[c] = to_bool(df[c]) if c in df else pd.array([pd.NA] * len(df), dtype="boolean")
report["is_forecast_unknown"] = int(df["is_forecast"].isna().sum())
df["is_forecast"] = df["is_forecast"].fillna(False)

df["opportunity_id"] = df["opportunity_id"].astype(str).str.strip()
dup = df.duplicated("opportunity_id", keep="first")
report["duplicate_ids_dropped"] = int(dup.sum())
df = df[~dup].copy()
df["grant_id"] = df["opportunity_id"]

df["summary_text"] = df["summary_description"].map(strip_html)
df["grant_text"] = (df["opportunity_title"] + "\n" + df["summary_text"]
                    + "\n" + df["applicant_eligibility_description"]).map(strip_html)

report["rows_out"] = len(df)
return df.sort_values("grant_id", kind="stable").reset_index(drop=True), report

def main() -> None:
    raw = pd.read_csv(config.RAW_GRANTS, dtype=str, keep_default_na=False)
    df, report = clean(raw)
    CLEANED.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(CLEANED, index=False)
    print(f"Layer 0: {report['rows_in']} rows in, {report['rows_out']} out "
          f"({report['duplicate_ids_dropped']} duplicate ids dropped) -> {CLEANED}")
    if report["unparseable"]:
        print(f" WARNING unparseable values (turned into blanks): {report['unparseable']}")
    if report["is_forecast_unknown"]:
        print(f" WARNING {report['is_forecast_unknown']} rows had no is_forecast value; treated as open")

if __name__ == "__main__":
    main()

```

src/deterministic.py

```
"""Layer 1 - deterministic checks. Plain pandas, no AI, no semantic judgment.

python src/clean_data.py && python src/deterministic.py

Reads data/processed/grants_cleaned.csv (Layer 0), agency/priorities.json
Writes data/processed/grants_features.csv <- input to Layers 2, 3 and 4
docs/layer1_report.md <- counts per flag, for auditing

Rules
* Dates use the fixed `as_of_date` from priorities.json, never today().
* Forecasts are a separate track (`track = forecast`). They have no close date by
  design, so they are never expired and never get a deadline score.
* Hard filters are conservative: a grant is only flagged `hard_filtered` when its
  close date is before as_of_date or its status says closed/archived/cancelled.
  Filtered rows STAY in the file (Layer 5 samples them to look for false negatives).
* Structured eligibility (who is listed as an applicant) is a signal, never a filter.
* Layer 4 needs two objective 0-10 scores from here: `financial_value_score`
  (always a number) and `deadline_score` (blank when there is no real deadline).
"""

import re

import numpy as np
import pandas as pd

import config
from clean_data import BOOL_COLS, CLEANED, DATE_COLS, NUM_COLS, to_bool
from text_utils import strip_html

REPORT = config.ROOT / "docs" / "layer1_report.md"

# Defaults mirror agency/priorities.json -> "layer1"; the JSON wins when both exist.
DEFAULTS = {
    "as_of_date": "2026-09-30",
    "data_pulled_date": "2026-08-18",
    "close_date_placeholder_year_min": 2090,
    "deadline_days": [7, 14, 30],
    "deadline_score_points": [[0, 0], [7, 2], [14, 4], [30, 7], [60, 10]],
    "large_award_usd": 1_000_000,
    "few_awards_max": 3,
    "min_description_chars": 100,
    "broad_applicant_types_min": 12,
    "financial_log_min_usd": 100_000,
    "financial_log_max_usd": 10_000_000,
    "financial_total_only_cap": 7.0,
    "financial_unknown_score": 5.0,
    "hard_filter_statuses": ["closed", "archived", "cancelled", "canceled"],
}

# Grants.gov applicant-type slugs, grouped into the signals we report.
STATE = {"state_governments"}
LOCAL = {"county_governments", "city_or_township_governments", "special_district_governments"}
HIGHER_ED = {"public_and_state_institutions_of_higher_education",
             "private_institutions_of_higher_education"}
NONPROFIT = {"nonprofits_non_higher_education_with_501c3",
             "nonprofits_non_higher_education_without_501c3"}
TRIBAL = {"federally_recognized_native_american_tribal_governments",
          "other_native_american_tribal_organizations"}
FOR_PROFIT = {"for_profit_organizations_other_than_small_businesses", "small_businesses"}
OTHER_KNOWN = {"individuals", "unrestricted", "other", "independent_school_districts",
              "public_and_indian_housing_authorities"}
KNOWN_SLUGS = STATE | LOCAL | HIGHER_ED | NONPROFIT | TRIBAL | FOR_PROFIT | OTHER_KNOWN

_COST_SHARE_WORDS = re.compile(
    r"cost[- ]?share|matching (funds|requirement|share|contribution)|match(ing)? (is )?required"
    r"|non-federal (share|match)", re.I)
```

```

_COST_SHARE_PCT = [
    re.compile(r"(\d{1,3}(?:\.\d+)?)\s*(?:%|percent)\s*(?:cost[- ]?shar\w*|match\w*|non-federal)", re.I),
    re.compile(r"(?:cost[- ]?shar\w*|match\w*)(^\.%\n){0,40}?(\\d{1,3}(?:\.\d+)?)\s*(?:%|percent)", re.I),
]
_STATE_TEXT = re.compile(r"state (government|agenc|department|health|public health)|departments? of health", re.I)
_INDIV_ONLY = re.compile(r"(only|solely|exclusively)[^\.]{0,40}individuals?|individuals?[^\.]{0,20}(only|solely)", re.I)
_HIGHER_ED_ONLY = re.compile(
    r"(only|limited to|restricted to|solely)[^\.]{0,80}(institutions? of higher education|universit|colleges?)", re.I)
_TRIBAL_ONLY = re.compile(r"(only|limited to|restricted to|solely)[^\.]{0,80}(tribal|tribe|native american)", re.I)
_ROLLING = re.compile(r"anytime|continuing basis|ongoing basis|year-round|rolling|no submission deadline"
    r"|accepted on a continu", re.I)
# Rolling grants whose money can run out are effectively first-come, first-served.
_FUNDS_LIMITED = re.compile(r"first[- ]come|until (all )?(available )?funds|funds (are|have been)
    (expended|exhausted)"
    r"|processed as (they are )?received|while funds last", re.I)
# Research and training mechanisms (NIH-style activity codes). A state DHHS cannot realistically
# lead these, but may partner with a university on them. Deliberately excludes U60/U58/U50
# (CDC state cooperative agreements), U13 (conferences) and U18 (e.g. FDA lab capacity grants).
_RESEARCH_CODE = re.compile(r"\b(R\d{2}|K\d{2}|F\d{2}|T\d{2}|P\d{2}|U01|U19|U24|U54|U2C|UG3|UH3|X01|DP[1-5])\b")
# Administrative notices, not funding opportunities: they exist only to move an existing award to a
# new organization (CDC "RFA-xx-18-000 ... Type 6 Applications", NIH successor-in-interest / Type 7).
_ADMIN_NOTICE = re.compile(r"type 6 application|successor[- ]in[- ]interest|change of recipient organization", re.I)
# Values Grants.gov uses as "no real number" in this export: $999,999,999 and 2^31 (a software
# maximum) as amounts, 9,999 as a number of awards. Read as money they would win "largest award".
PLACEHOLDER_AMOUNTS = {999_999_999.0, 2_147_483_647.0, 2_147_483_648.0}
PLACEHOLDER_AWARD_COUNTS = {9_999.0}

```

— Loading —

```

def settings(agency: dict) -> dict:
    """Defaults, overridden by priorities.json -> layer1, plus the fixed as_of_date."""
    cfg = {**DEFAULTS, **{k: v for k, v in (agency.get("layer1") or {}).items() if not k.startswith("_")}}
    cfg["as_of_date"] = agency.get("as_of_date", cfg["as_of_date"])
    return cfg

```

```

def coerce_types(df: pd.DataFrame) -> pd.DataFrame:
    """Re-type grants_cleaned.csv after reading it back as strings."""
    df = df.copy()
    for c in DATE_COLS:
        df[c] = pd.to_datetime(df[c], errors="coerce")
    for c in NUM_COLS:
        df[c] = pd.to_numeric(df[c], errors="coerce")
    for c in BOOL_COLS:
        df[c] = to_bool(df[c])
    df["is_forecast"] = df["is_forecast"].fillna(False)
    return df

```

```

def load_cleaned() -> pd.DataFrame:
    return coerce_types(pd.read_csv(CLEANED, dtype=str, keep_default_na=False))

```

— Helpers —

```

def _matches(series: pd.Series, pattern: re.Pattern) -> pd.Series:
    return series.map(lambda t: bool(pattern.search(t)))

```

```

def parse_cost_share_percent(text: str) -> float:
    for pattern in _COST_SHARE_PCT:
        m = pattern.search(text)
        if m and 0 < float(m.group(1)) <= 100:
            return float(m.group(1))
    return float("nan")

```

— Feature groups —

```

def add_deadline_features(df: pd.DataFrame, cfg: dict) -> None:
    as_of = pd.Timestamp(cfg["as_of_date"])
    fc = df["is_forecast"].astype(bool)
    is_open = ~fc

    # Placeholder dates such as 2099-01-01 mean "no real deadline given".
    placeholder = df["close_date"].dt.year >= cfg["close_date_placeholder_year_min"]
    real_close = df["close_date"].where(~placeholder)
    df["close_date_placeholder"] = is_open & placeholder
    df["close_date_real"] = real_close.where(is_open)

    days = (real_close - as_of).dt.days
    df["days_until_close"] = days.where(is_open)
    df["missing_deadline"] = is_open & real_close.isna()
    df["expired"] = is_open & (real_close < as_of) # closing ON as_of is still open
    live = is_open & real_close.notna() & ~df["expired"]
    d7, d14, d30 = cfg["deadline_days"]
    df["deadline_under_7_days"] = live & (days <= d7)
    df["deadline_under_14_days"] = live & (days <= d14)
    df["deadline_under_30_days"] = live & (days <= d30)
    df["deadline_comfortable"] = live & (days > d30)

    # Layer 4 input (0-10). Blank when there is no real deadline or the grant is a forecast.
    xs, ys = zip(*cfg["deadline_score_points"])
    score = pd.Series(np.interp(days.fillna(0), xs, ys), index=df.index)
    df["deadline_score"] = score.where(is_open & real_close.notna()).round(2)

    # Forecast track: when might it open? Never hard-filtered.
    fpost, fclose = df["forecasted_post_date"], df["forecasted_close_date"]
    df["forecast_missing_post_date"] = fc & fpost.isna()
    df["days_until_forecast_post"] = (fpost - as_of).dt.days.where(fc)
    df["forecast_post_passed"] = fc & (fpost < as_of)
    df["forecast_close_passed"] = fc & (fclose < as_of)
    # Post date has passed but the forecast close date has not: it may already be open
    # on Grants.gov, so the watchlist should say "check now".
    df["forecast_check_now"] = df["forecast_post_passed"] & ~df["forecast_close_passed"]
    df["archive_date_passed"] = df["archive_date"] < as_of
    # "Proposals accepted anytime": no close date, but not an unknown deadline either.
    df["rolling_deadline"] = df["missing_deadline"] & df["close_date_description"].fillna("").map(
        lambda t: bool(ROLLING.search(str(t))))
    # ...but "until funds are expended" / "processed as received" means no real flexibility.
    rolling_text = df["close_date_description"].fillna("") + " " + df["summary_description"].fillna("")
    df["rolling_funds_limited"] = df["rolling_deadline"] & rolling_text.map(lambda t: bool(FUNDS_LIMITED.search(t)))

def add_funding_features(df: pd.DataFrame, cfg: dict) -> None:
    three = ["award_ceiling", "estimated_total_program_funding", "expected_number_of_awards"]
    # A 0 in these fields is a placeholder, not a real amount; so are the sentinel values.
    df["funding_zero_placeholder"] = (df[three] == 0).any(axis=1)
    sentinel = (df[["award_floor", "award_ceiling",
"estimated_total_program_funding"]].isin(PLACEHOLDER_AMOUNTS).any(axis=1)
| df["expected_number_of_awards"].isin(PLACEHOLDER_AWARD_COUNTS))
    df["funding_sentinel_placeholder"] = sentinel
    for c in ["award_floor", "award_ceiling", "estimated_total_program_funding"]:
        df[c] = df[c].where(~df[c].isin(PLACEHOLDER_AMOUNTS))
    df["expected_number_of_awards"] = df["expected_number_of_awards"].where(
        ~df["expected_number_of_awards"].isin(PLACEHOLDER_AWARD_COUNTS))
    for c in three:
        df[c] = df[c].where(df[c] > 0)

    ceiling, total, n_awards = df["award_ceiling"], df["estimated_total_program_funding"],
df["expected_number_of_awards"]
    avg = total / n_awards
    df["average_possible_award"] = avg
    df["financial_basis"] = np.select(
        [ceiling.notna(), avg.notna(), total.notna()],
        ["award_ceiling", "average_award", "program_total"], "unknown")
    amount = ceiling.fillna(avg).fillna(total)
    df["award_value_usd"] = amount

```

```

df["funding_known"] = amount.notna()
df["missing_award_amount"] = amount.isna()
df["large_award"] = amount >= cfg["large_award_usd"]
df["few_awards"] = n_awards.between(1, cfg["few_awards_max"])
df["award_floor_gt_ceiling"] = df["award_floor"] > ceiling

# Layer 4 input (0-10): log scale so a $50M award cannot drown out a better-fitting $1M one.
lo, hi = np.log10(cfg["financial_log_min_usd"]), np.log10(cfg["financial_log_max_usd"])
score = (10 * (np.log10(amount) - lo) / (hi - lo)).clip(0, 10)
# Total program funding is not what one applicant receives, so it earns less credit.
score = score.where(df["financial_basis"] != "program_total", score.clip(upper=cfg["financial_total_only_cap"]))
df["financial_value_imputed"] = amount.isna()
df["financial_value_score"] = score.fillna(cfg["financial_unknown_score"]).round(2)

def add_cost_share_features(df: pd.DataFrame) -> None:
    text = df["grant_text"]
    mention = _matches(text, _COST_SHARE_WORDS)
    df["cost_share_text_mention"] = mention
    df["cost_share_percent"] = text.map(parse_cost_share_percent)

    structured = df["is_cost_sharing"] # True / False / <NA>
    is_true = structured.fillna(False).astype(bool)
    is_false = ~structured.fillna(True).astype(bool)
    # A blank structured field is Unknown, never False (text can upgrade it to True).
    df["requires_cost_share"] = np.select(
        [is_true, is_false, mention.to_numpy()], ["True", "False", "True"], "Unknown")
    df["cost_share_conflict"] = is_false & df["cost_share_percent"].notna()

def add_eligibility_features(df: pd.DataFrame, cfg: dict) -> None:
    """Signals only. None of these may be used to drop a grant."""
    slugs = df["applicant_types"].map(lambda s: {x.strip() for x in s.split(";") if x.strip()})
    has = lambda names: slugs.map(lambda s: bool(s & names))
    df["state_government_listed"] = has(STATE)
    df["local_government_listed"] = has(LOCAL)
    df["universities_listed"] = has(HIGHER_ED)
    df["nonprofits_listed"] = has(NONPROFIT)
    df["tribal_listed"] = has(TRIBAL)
    df["for_profit_listed"] = has(FOR_PROFIT)
    df["individuals_listed"] = has({"individuals"})
    df["unrestricted_listed"] = has({"unrestricted"})
    df["other_listed"] = has({"other"})
    df["n_applicant_types"] = slugs.map(len)
    df["applicant_types_blank"] = df["n_applicant_types"] == 0
    # Listing nearly every type (e.g. NIH research grants) says little about DHHS specifically.
    df["broad_eligibility"] = df["n_applicant_types"] >= cfg["broad_applicant_types_min"]
    df["unknown_applicant_slugs"] = slugs.map(lambda s: ";".join(sorted(s - KNOWN_SLUGS)))

    elig = df["applicant_eligibility_description"].map(strip_html)
    df["text_mentions_state_agency"] = _matches(elig, _STATE_TEXT)
    df["text_individuals_only"] = _matches(elig, _INDIV_ONLY)
    df["text_higher_ed_only"] = _matches(elig, _HIGHER_ED_ONLY)
    df["text_tribal_only"] = _matches(elig, _TRIBAL_ONLY)

def add_listing_and_quality_features(df: pd.DataFrame, cfg: dict) -> None:
    # "93.243|Name;16.590|Name" -> ["93.243", "16.590"]
    codes = df["opportunity_assistance_listings"].map(
        lambda s: [x.split("|")[0].strip() for x in s.split(";") if x.strip()])
    df["n_assistance_listings"] = codes.map(len)
    df["aln_is_hhs"] = codes.map(lambda cs: any(c.startswith("93.") for c in cs))
    agency_code = df["agency_code"] if "agency_code" in df else pd.Series("", index=df.index)
    # ...plus the bare CDC placeholders titled only "RFA-XX-18-000" ("Submit application as necessary.").
    df["administrative_notice"] = (df["opportunity_title"] + " " + df["summary_text"]).map(
        lambda t: bool(_ADMIN_NOTICE.search(str(t))) | df["opportunity_title"].str.strip().str.fullmatch(
            r"RFA-[A-Z]{2}-\d{2}-000")
    # Research mechanism: an NIH opportunity, or a research/training activity code in the title.
    is_nih = agency_code.str.upper().str.startswith("HHS-NIH")
    code = df["opportunity_title"].map(lambda t: (_RESEARCH_CODE.search(str(t)) or [None])[0])

```

```

df["research_mechanism"] = is_nih | code.notna()
df["research_mechanism_reason"] = np.where(code.notna(), "activity code " + code.fillna(""),
                                             np.where(is_nih, "NIH opportunity", ""))

df["federal_agency_is_hhs"] = (
    df["top_level_agency_name"].eq("Department of Health and Human Services")
    | agency_code.str.upper().str.startswith("HHS"))

df["missing_description"] = df["summary_text"].str.len() == 0
df["description_too_short"] = df["summary_text"].str.len() < cfg["min_description_chars"]
df["missing_eligibility_description"] = df["applicant_eligibility_description"].map(strip_html).str.len() == 0

# Same title + same agency under a different id: a repost or an amendment. A feature only.
key = (df["opportunity_title"].str.lower().str.replace(r"\s+", " ", regex=True).str.strip()
      + "|" + df["agency_name"].str.lower().str.strip())
order = df.assign(_key=key).sort_values(["_key", "post_date", "grant_id"], kind="stable")
first = order.groupby("_key")["grant_id"].transform("first")
df["possible_duplicate_of"] = first.where(first != order["grant_id"], "").reindex(df.index)

def add_hard_filters(df: pd.DataFrame, cfg: dict) -> None:
    status = df["opportunity_status"].str.lower()
    bad_status = status.isin(cfg["hard_filter_statuses"])
    # A posted grant past its archive date no longer accepts applications, even when the
    # export has no close date (12 such grants say "Funding Opportunity is Archived").
    archived = ~df["is_forecast"] & df["archive_date_passed"] & ~df["expired"]
    reason = np.select([df["expired"], archived, bad_status, df["administrative_notice"]],
                      ["closed_before_as_of_date", "archived_before_as_of_date",
                       ("status_" + status).to_numpy(), "administrative_award_transfer_notice"], "")
    df["hard_filter_reason"] = reason
    df["hard_filtered"] = df["hard_filter_reason"] != ""

# — Build, check, report —————

FRONT_COLUMNS = ["grant_id", "opportunity_title", "track", "is_forecast", "hard_filtered", "hard_filter_reason"]

def build_features(cleaned: pd.DataFrame, cfg: dict) -> pd.DataFrame:
    """Typed Layer 0 frame in, feature frame out (same rows, sorted by grant_id)."""
    df = cleaned.sort_values("grant_id", kind="stable").reset_index(drop=True)
    df["is_forecast"] = df["is_forecast"].astype(bool)
    df["track"] = np.where(df["is_forecast"], "forecast", "open")
    add_deadline_features(df, cfg)
    add_funding_features(df, cfg)
    add_cost_share_features(df)
    add_eligibility_features(df, cfg)
    add_listing_and_quality_features(df, cfg)
    add_hard_filters(df, cfg)
    df["as_of_date"] = cfg["as_of_date"]
    df["data_pulled_date"] = cfg["data_pulled_date"]
    rest = [c for c in df.columns if c not in FRONT_COLUMNS]
    return df[FRONT_COLUMNS + rest]

def check(n_cleaned: int, df: pd.DataFrame) -> None:
    """Fail loudly if Layer 1 would break the layers that read its output."""
    from semantic_triage import REQUIRED_COLUMNS # Layer 2's contract

    assert len(df) == n_cleaned, "Layer 1 must not add or drop rows"
    assert df["grant_id"].is_unique, "grant_id must be unique"
    missing = [c for c in REQUIRED_COLUMNS if c not in df]
    assert not missing, f"missing columns Layer 2 needs: {missing}"
    assert (df["track"] == "forecast").equals(df["is_forecast"]), "track must mirror is_forecast"
    assert not (df["is_forecast"] & (df["expired"] | (df["hard_filter_reason"] ==
"closed_before_as_of_date"))).any(), \
        "a forecast was treated as expired"
    assert df["financial_value_score"].between(0, 10).all(), "financial_value_score must be 0-10 and never blank"
    scored = df["deadline_score"].dropna()
    assert scored.between(0, 10).all(), "deadline_score must be 0-10"
    assert df.loc[df["is_forecast"], "deadline_score"].isna().all(), "forecasts must have no deadline_score"

```

```

assert df.loc[~df["is_forecast"] & ~df["missing_deadline"], "deadline_score"].notna().all()

def _counts(series: pd.Series) -> str:
    return "\n".join(f"- {k}: {v}" for k, v in series.value_counts().items()) or "- none"

def report_markdown(df: pd.DataFrame, cfg: dict) -> str:
    n = lambda mask: int(mask.sum())
    op, fc = df[~df["is_forecast"]], df[df["is_forecast"]]
    kept = df[~df["hard_filtered"]]
    flags = ["deadline_under_7_days", "deadline_under_14_days", "deadline_under_30_days", "deadline_comfortable"]
    unknown = sorted({s for v in df["unknown_applicant_slugs"] for s in v.split(";") if s})
    lines = [
        "# Layer 1 report", "",
        f"Generated by `src/deterministic.py`. All date math uses the fixed as-of date "
        f"**{cfg['as_of_date']}**. The data was pulled **{cfg['data_pulled_date']}**, so anything "
        "posted or changed after that is not in it.", "",
        "## Rows", f"- total: {len(df)}", f"- open track: {len(op)}", f"- forecast track: {len(fc)}",
        f"- sent on to Layer 2 (not hard-filtered): {len(kept)}", "",
        "## Hard filters (rows stay in the file, flagged)", _counts(df["hard_filter_reason"].replace("", "kept")),
        "",
        "## Open grants: deadlines",
        f"- expired (closed before as-of date): {n(op['expired'])}",
        f"- rolling (\\`accepted anytime\\`): {n(op['rolling_deadline'])}, of which first-come / until funds run out: "
        f"{n(op['rolling_funds_limited'])}",
        f"- no close date in the data: {n(op['missing_deadline'])} "
        f"(of which {n(op['close_date_placeholder'])} carry a placeholder year-
{cfg['close_date_placeholder_year_min']}+ date)",
        *[f"- {f}: {n(op[f])}" for f in flags], "",
        "## Forecast grants (watchlist track)",
        f"- forecasted post date already passed: {n(fc['forecast_post_passed'])}",
        f"- forecasted close date already passed (stale?): {n(fc['forecast_close_passed'])}",
        f"- post date passed but close not: {n(fc['forecast_check_now'])} (may already be open, check Grants.gov
now)",
        f"- no forecasted post date: {n(fc['forecast_missing_post_date'])}", "",
        "## Funding",
        _counts(df["financial_basis"]),
        f"- rows where a 0 amount was treated as a placeholder: {n(df['funding_zero_placeholder'])}",
        f"- rows with a sentinel amount ($999,999,999, 2^31, 9,999 awards) set to missing:
{n(df['funding_sentinel_placeholder'])}",
        f"- financial_value_score imputed to {cfg['financial_unknown_score']} (no amount at all):
{n(df['financial_value_imputed'])}",
        f"- award_floor above ceiling: {n(df['award_floor_gt_ceiling'])}", "",
        "## Cost sharing (structured field, text can upgrade Unknown)", _counts(df["requires_cost_share"]),
        f"- structured says False but the text states a percentage: {n(df['cost_share_conflict'])}", "",
        "## Eligibility signals (never used to filter)",
        f"- state_governments listed: {n(df['state_government_listed'])}",
        f"- broad eligibility ({cfg['broad_applicant_types_min']}+ types listed): {n(df['broad_eligibility'])}",
        f"- applicant types blank: {n(df['applicant_types_blank'])}",
        f"- eligibility text blank: {n(df['missing_eligibility_description'])}",
        f"- state grants but text says higher-ed / tribal / individuals only: "
        f"{n(df['state_government_listed'] & (df['text_higher_ed_only'] | df['text_tribal_only'] |
df['text_individuals_only']))}",
        f"- applicant slugs not in our known list: {unknown or 'none'}", "",
        "## Research mechanisms (moved to the research-partnerships list in Layer 4)",
        f"- research mechanism: {n(df['research_mechanism'])} (of which still open or forecast:
{n(kept['research_mechanism'])})", "",
        "## HHS signals",
        f"- assistance listing 93.x: {n(df['aln_is_hhs'])}",
        f"- federal agency is HHS: {n(df['federal_agency_is_hhs'])}", "",
        "## Data quality",
        f"- description under {cfg['min_description_chars']} characters: {n(df['description_too_short'])} "
        f"(blank: {n(df['missing_description'])})",
        f"- same title + agency under a different id: {n(df['possible_duplicate_of'] != '')}",
        f"- open grants whose archive date has passed: {n(op['archive_date_passed'])}", "",
        "## Layer 4 inputs",
        f"- financial_value_score: mean {df['financial_value_score'].mean():.2f}, "
        f"{n(df['financial_value_score'] >= 8)} grants score 8 or more",
        f"- deadline_score: present for {int(df['deadline_score'].notna().sum())} open grants, "

```

```

        f"blank for forecasts and for the {n(op['missing_deadline'])} with no real close date", "",
    ]
    return "\n".join(lines)

def main() -> None:
    cfg = settings(config.load_agency())
    cleaned = load_cleaned()
    features = build_features(cleaned, cfg)
    check(len(cleaned), features)

    config.LAYER1_OUTPUT.parent.mkdir(parents=True, exist_ok=True)
    features.to_csv(config.LAYER1_OUTPUT, index=False)
    REPORT.parent.mkdir(parents=True, exist_ok=True)
    REPORT.write_text(report_markdown(features, cfg), encoding="utf-8")
    print(f"Layer 1: {len(features)} rows -> {config.LAYER1_OUTPUT}")
    print(f"  open={int((~features['is_forecast']).sum())} forecast={int(features['is_forecast'].sum())} "
          f"hard_filtered={int(features['hard_filtered'].sum())} -> report: {REPORT}")

if __name__ == "__main__":
    main()

```

src/semantic_triage.py

```
"""Layer 2 - fast semantic triage with Jev.

For every grant that survived Layer 1's hard filters, Jev answers four `choice`
questions (prompts/jev_triage.json): domain relevance, matched strategic goal,
applicant role, owning division. Jev returns a probability for every option, and
those probabilities are the confidence signal: a grant is deprioritized only when
Jev puts high probability on "irrelevant" or "ineligible"; everything else goes to
Layer 3. Deprioritized grants are kept (not deleted) so validation can sample them.

Usage:
    python src/semantic_triage.py          # real run (needs AI_GATEWAY_API_KEY, or a full cache)
    python src/semantic_triage.py --mock   # dry run with fake answers
    python src/semantic_triage.py --limit 20 # first 20 grants only
"""

import argparse
import copy
import json
import sys
from concurrent.futures import ThreadPoolExecutor

import pandas as pd

import config
from clients import JevClient, MockJevClient
from llm_cache import ResponseCache, prompt_version, request_hash
from text_utils import strip_html, truncate

# Columns Layer 2 needs from Layer 1's output (data/processed/grants_features.csv).
REQUIRED_COLUMNS = [
    "grant_id", "opportunity_title", "agency_name", "summary_description",
    "applicant_types", "applicant_eligibility_description",
    "opportunity_assistance_listings", "funding_instruments",
    "is_forecast", "hard_filtered",
]

RELEVANCE_VALUE = {"none": 0.0, "weak": 1 / 3, "moderate": 2 / 3, "strong": 1.0}
MAX_SUMMARY_CHARS = 3000

# — Prompt construction —————

def load_prompt(agency: dict) -> tuple[str, dict, str]:
    """-> (context sentence, Jev question dict, prompt version hash)."""
    path = config.PROMPTS / "jev_triage.json"
    raw_text = path.read_text(encoding="utf-8")
    spec = json.loads(raw_text)
    context = spec["context"].format(agency_name=agency["agency_name"],
                                     mission=agency["mission"], user=agency["user"])
    questions = copy.deepcopy(spec["questions"])
    for q in questions.values():
        if q["criteria"] == "FROM_CONFIG:strategic_priorities":
            q["id"] = {p["id"]: f"{p['name']}: {p['description']}"
                       for p in agency["strategic_priorities"]}
            q["criteria"]["none"] = "Advances none of these goals."
        elif q["criteria"] == "FROM_CONFIG:divisions":
            q["id"] = {d["id"]: f"{d['name']} ({d['scope']})" for d in agency["divisions"]}
            q["criteria"]["none"] = "No division of this agency would own it."
    version = prompt_version(raw_text, json.dumps(config.prompt_fields(agency), sort_keys=True))
    return context, questions, version

def grant_state(row: pd.Series, context: str) -> dict:
    """The structured `state` Jev reasons over."""
    return {
        "task": context,
        "grant": {
```

```

        "title": str(row["opportunity_title"]),
        "federal_agency": str(row["agency_name"]),
        "assistance_listings": str(row.get("opportunity_assistance_listings", "") or ""),
        "funding_instruments": str(row.get("funding_instruments", "") or ""),
        "applicant_types": str(row.get("applicant_types", "") or "").replace(";", ", ", " "),
        "eligibility_text": strip_html(row.get("applicant_eligibility_description", "")) or "(none given)",
        "summary": truncate(strip_html(row.get("summary_description", "")), MAX_SUMMARY_CHARS) or "(none given)",
    },
}

```

— Confidence-aware routing —————

```

def _probs(answers: dict, question: str) -> dict:
    return (answers.get(question) or {}).get("probabilities") or {}

def route(answers: dict, thresholds: dict) -> tuple[str, str]:
    """-> ("deep_review" | "deprioritized", reason). Pure function; unit-tested."""
    rel = _probs(answers, "domain_relevance")
    role = _probs(answers, "applicant_role")
    if not rel:
        return "deep_review", "missing Jev relevance answer"
    p_none = rel.get("none", 0.0)
    p_none_weak = p_none + rel.get("weak", 0.0)
    p_inelig = role.get("ineligible", 0.0)
    if p_none >= thresholds["deprioritize_if_p_none_at_least"]:
        return "deprioritized", f"P(unrelated)={p_none:.2f}"
    if p_none_weak >= thresholds["deprioritize_if_p_none_or_weak_at_least"]:
        return "deprioritized", f"P(unrelated or weak)={p_none_weak:.2f}"
    if p_inelig >= thresholds["deprioritize_if_p_ineligible_at_least"]:
        return "deprioritized", f"P(ineligible)={p_inelig:.2f}"
    return "deep_review", "not confidently irrelevant"

def summarize(answers: dict) -> dict:
    """Flatten Jev's answers into CSV columns."""
    out = {}
    for q in ("domain_relevance", "matched_goal", "applicant_role", "owning_division"):
        a = answers.get(q) or {}
        probs = a.get("probabilities") or {}
        choice = a.get("choice") or (max(probs, key=probs.get) if probs else None)
        out[q] = choice
        out[f"{q}_p"] = round(probs.get(choice, 0.0), 4) if choice else None
        out[f"{q}_probs"] = json.dumps({k: round(v, 4) for k, v in probs.items()})
    rel = _probs(answers, "domain_relevance")
    role = _probs(answers, "applicant_role")
    expected_rel = sum(RELEVANCE_VALUE.get(k, 0) * v for k, v in rel.items())
    p_ours = role.get("lead", 0.0) + role.get("partner", 0.0)
    out["expected_relevance"] = round(expected_rel, 4)
    out["p_lead_or_partner"] = round(p_ours, 4)
    # Orders the Layer 3 queue only; the final ranking is Layer 4's job.
    out["triage_priority"] = round(expected_rel * (0.5 + 0.5 * p_ours), 4)
    return out

```

— Runner —————

```

def load_grants() -> pd.DataFrame:
    if not config.LAYER1_OUTPUT.exists():
        import clean_data
        import deterministic
        clean_data.main()
        deterministic.main()
    df = pd.read_csv(config.LAYER1_OUTPUT)
    source = config.LAYER1_OUTPUT
    missing = [c for c in REQUIRED_COLUMNS if c not in df.columns]
    if missing:
        sys.exit(f"Layer 1 output is missing columns Layer 2 needs: {missing}")
    print(f"Layer 2 input: {len(df)} grants from {source}")

```

```

return df

def run(mock: bool = False, limit: int | None = None, workers: int = 8) -> pd.DataFrame:
    agency = config.load_agency()
    context, questions, version = load_prompt(agency)
    client = MockJevClient() if mock else JevClient()
    cache = ResponseCache(config.MOCK_CACHE if mock else config.CACHE, "layer2_jev")
    out_dir = config.MOCK_RESULTS if mock else config.RESULTS
    out_dir.mkdir(parents=True, exist_ok=True)

    df = load_grants()
    todo = df[~df["hard_filtered"].astype(bool)]
    if limit:
        todo = todo.head(limit)
    print(f"Triaging {len(todo)} grants (skipping {len(df) - len(todo)} hard-filtered or beyond --limit)")

    def triage(row) -> dict:
        state = grant_state(row, context)
        key = request_hash(client.model, version, {"state": state, "questions": questions})
        answers = cache.get(row["grant_id"], key)
        cached = answers is not None
        if not cached:
            answers = client.evaluate(state, questions)
            cache.put(row["grant_id"], key, client.model, version, answers)
        decision, reason = route(answers, agency["layer2_routing"])
        return {"grant_id": row["grant_id"], "opportunity_title": row["opportunity_title"],
                "track": "forecast" if str(row["is_forecast"]).lower() == "true" else "open",
                **summarize(answers, "route": decision, "route_reason": reason,
                           "model": client.model, "prompt_version": version, "from_cache": cached)}

    rows, errors = [], []
    with ThreadPoolExecutor(max_workers=workers) as pool:
        futures = {pool.submit(triage, r): r["grant_id"] for _, r in todo.iterrows()}
        for i, fut in enumerate(futures, 1):
            try:
                rows.append(fut.result())
            except Exception as e: # one bad grant must not kill the run
                errors.append({"grant_id": futures[fut], "error": str(e)[:300]})
            if i % 100 == 0:
                print(f" {i}/{len(futures)}")

    result = pd.DataFrame(rows)
    result.to_csv(out_dir / "jev_outputs.csv", index=False)
    if errors:
        pd.DataFrame(errors).to_csv(out_dir / "jev_errors.csv", index=False)
        print(f"WARNING: {len(errors)} grants failed; see {out_dir / 'jev_errors.csv'}. "
              f"They are NOT in jev_outputs.csv - rerun to retry them.")
    if len(result):
        print(result["route"].value_counts().to_string())
        print("By track:\n" + pd.crosstab(result["track"], result["route"]).to_string())
    print(f"Wrote {out_dir / 'jev_outputs.csv'}" + (" [MOCK - fake answers]" if mock else ""))
    return result

if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--mock", action="store_true", help="fake Jev answers; no API key needed")
    ap.add_argument("--limit", type=int, help="only triage the first N grants")
    ap.add_argument("--workers", type=int, default=8)
    a = ap.parse_args()
    run(mock=a.mock, limit=a.limit, workers=a.workers)

```

src/deep_analysis.py

```
"""Layer 3 - deep evaluation with Muse Spark 1.3 (via the Vercel AI Gateway).

Runs only on grants Layer 2 routed to "deep_review". For each one the model returns
structured judgments plus verbatim evidence quotes (prompts/deep_analysis.txt).
Python then checks the model's work:

    * every quote must appear verbatim in the grant text; an unverified eligibility
      quote downgrades the applicant role to "unclear";
    * ids must exist in the agency config, scores must be in range;
    * Layer 2 (Jev) and Layer 3 answers are compared, and disagreements are logged.

Every caught problem goes to data/results/ai_error_log.csv - the raw material for
"one place the AI was wrong and how we caught it".

Deadlines and award size are NOT judged here; they are deterministic (Layers 1/4).

Usage:
    python src/deep_analysis.py          # real run (needs AI_GATEWAY_API_KEY, or a full cache)
    python src/deep_analysis.py --mock  # dry run on mock Layer 2 output
"""

import argparse
import json
import re
from concurrent.futures import ThreadPoolExecutor

import pandas as pd

import config
from clients import ChatClient, MockChatClient
from llm_cache import ResponseCache, prompt_version, request_hash
from semantic_triage import load_grants
from text_utils import quote_in_source, strip_html, truncate

ROLES = {"lead", "partner", "atypical", "ineligible", "unclear"}
MAX_SUMMARY_CHARS = 8000
MAX_ELIGIBILITY_CHARS = 3000

# — Prompt —————

def load_template() -> tuple[str, str, str]:
    raw = (config.PROMPTS / "deep_analysis.txt").read_text(encoding="utf-8")
    system, user = raw.split("=== USER ===")
    system = system.replace("=== SYSTEM ===", "").strip()
    return system, user.strip(), raw

def agency_blocks(agency: dict) -> dict:
    pri = []
    for p in agency["strategic_priorities"]:
        pri.append(f"{p['id']} {p['name']}: {p['description']}")
        pri += [f" {o['id']}: {o['text']}" for o in p["objectives"]]
    div = [f"{d['id']}: {d['name']} ({d['scope']}" for d in agency["divisions"]]
    return {"agency_name": agency["agency_name"], "mission": agency["mission"],
            "user": agency["user"], "priorities_block": "\n".join(pri),
            "divisions_block": "\n".join(div)}

def grant_texts(row: pd.Series) -> dict:
    return {
        "title": str(row["opportunity_title"]),
        "federal_agency": str(row["agency_name"]),
        "assistance_listings": str(row.get("opportunity_assistance_listings", "") or "(none)"),
        "funding_instruments": str(row.get("funding_instruments", "") or "(none)"),
        "applicant_types": str(row.get("applicant_types", "") or "(none)").replace(";", ", "),
    }
```

```

        "summary": truncate(strip_html(row.get("summary_description", "")), MAX_SUMMARY_CHARS) or "(none given)",
        "eligibility": truncate(strip_html(row.get("applicant_eligibility_description", "")),
                                MAX_ELIGIBILITY_CHARS) or "(none given)",
    }

```

— Parsing and validation

```

def parse_json(text: str) -> dict:
    """Models sometimes wrap JSON in ``fences`` or add a sentence; take the object."""
    m = re.search(r"\{.*\}", text, re.S)
    if not m:
        raise ValueError("no JSON object in the response")
    return json.loads(m.group(0))

def validate(out: dict, agency: dict) -> list[str]:
    """-> list of problems (empty = valid)."""
    problems = []
    goal_ids = {p["id"] for p in agency["strategic_priorities"]} | {"none"}
    obj_ids = config.objective_ids(agency) | {"none"}
    div_ids = {d["id"] for d in agency["divisions"]} | {"none"}
    for k in ("strategic_alignment", "operational_fit"):
        v = out.get(k)
        if not isinstance(v, (int, float)) or isinstance(v, bool) or not 0 <= v <= 10:
            problems.append(f"{k} must be a number 0-10, got {v!r}")
    if out.get("matched_goal") not in goal_ids:
        problems.append(f"matched_goal {out.get('matched_goal')!r} is not a goal id")
    if out.get("matched_objective") not in obj_ids:
        problems.append(f"matched_objective {out.get('matched_objective')!r} is not an objective id")
    elif (out.get("matched_objective") != "none"
          and not str(out["matched_objective"]).startswith(f"{out.get('matched_goal')}.")):
        problems.append(f"objective {out['matched_objective']} does not belong to goal {out.get('matched_goal')}")
    if out.get("applicant_role") not in ROLES:
        problems.append(f"applicant_role {out.get('applicant_role')!r} not in {sorted(ROLES)}")
    if out.get("owning_division") not in div_ids:
        problems.append(f"owning_division {out.get('owning_division')!r} is not a division id")
    if not isinstance(out.get("recommended_for_review"), bool):
        problems.append("recommended_for_review must be true/false")
    if not isinstance(out.get("restrictions", []), list):
        problems.append("restrictions must be a list")
    for k in ("strategic_reason", "one_line_rationale", "major_risk"):
        if not isinstance(out.get(k), str) or not out[k].strip():
            problems.append(f"{k} is missing")
    return problems

```

— Checks on the model's claims

```

def verify(out: dict, texts: dict) -> tuple[dict, list[dict]]:
    """Check quotes against the source. -> (verified fields, error-log entries)."""
    errors = []
    strategic_src = f"{texts['title']}\n{texts['summary']}"
    elig_src = f"{texts['eligibility']}\n{texts['summary']}\n{texts['applicant_types']}"

    s_ok = quote_in_source(out.get("strategic_evidence_quote"), strategic_src)
    e_quote = out.get("eligibility_evidence_quote") or ""
    e_ok = quote_in_source(e_quote, elig_src)

    role = out.get("applicant_role")
    role_verified = role
    if not s_ok:
        errors.append({"check": "unverified_strategic_quote", "severity": "high",
                       "detail": f"quote not found in grant text: {str(out.get('strategic_evidence_quote'))[:200]!r}"})
    if role in ("lead", "partner") and not e_ok:
        role_verified = "unclear"
        errors.append({"check": "unverified_eligibility_quote", "severity": "high",
                       "detail": f"claimed role '{role}' but its quote was not found in the eligibility text "
                                f"({e_quote[:200]!r}); role downgraded to 'unclear'"})

```

```

elif e_quote and not e_ok:
    errors.append({"check": "unverified_eligibility_quote", "severity": "medium",
                  "detail": f"quote not found: {e_quote[:200]!r}"})
return ({ "strategic_quote_verified": s_ok, "eligibility_quote_verified": e_ok,
         "applicant_role_verified": role_verified}, errors)

def compare_layers(l2: pd.Series, out: dict) -> list[dict]:
    """Log where Jev (Layer 2) and the deep model (Layer 3) disagree."""
    found = []
    rel, align = l2.get("domain_relevance"), out.get("strategic_alignment", 0)
    if rel == "strong" and align <= 3:
        found.append({"check": "layer2_vs_layer3_relevance", "severity": "medium",
                     "detail": f"Jev said 'strong' (p={l2.get('domain_relevance_p')}) but deep model scored
alignment {align}/10"})
    if rel in ("none", "weak") and align >= 7:
        found.append({"check": "layer2_vs_layer3_relevance", "severity": "high",
                     "detail": f"Jev said '{rel}' (p={l2.get('domain_relevance_p')}) but deep model scored alignment
{align}/10 "
                     f"- a possible Layer 2 false negative"})
    r2, r3 = l2.get("applicant_role"), out.get("applicant_role")
    ours, not_ours = {"lead", "partner"}, {"atypical", "ineligible"}
    if (r2 in ours and r3 in not_ours) or (r2 in not_ours and r3 in ours):
        found.append({"check": "layer2_vs_layer3_role", "severity": "medium",
                     "detail": f"Jev role '{r2}' vs deep model role '{r3}'"})
    if l2.get("owning_division") and out.get("owning_division") and l2["owning_division"] != out["owning_division"]:
        found.append({"check": "layer2_vs_layer3_division", "severity": "low",
                     "detail": f"Jev division {l2['owning_division']} vs deep model {out['owning_division']}"})
    return found

# — Runner —————

def run(mock: bool = False, limit: int | None = None, workers: int = 4) -> pd.DataFrame:
    agency = config.load_agency()
    system, user_tmpl, raw_prompt = load_template()
    blocks = agency_blocks(agency)
    version = prompt_version(raw_prompt, json.dumps(config.prompt_fields(agency), sort_keys=True))
    client = MockChatClient() if mock else ChatClient()
    cache = ResponseCache(config.MOCK_CACHE if mock else config.CACHE, "layer3_deep")
    out_dir = config.MOCK_RESULTS if mock else config.RESULTS

    l2 = pd.read_csv(out_dir / "jev_outputs.csv")
    queue = l2[l2["route"] == "deep_review"].sort_values("triage_priority", ascending=False)
    cap = limit or agency.get("layer3", {}).get("max_grants")
    overflow = queue.iloc[cap:] if cap else queue.iloc[0:0]
    queue = queue.iloc[:cap] if cap else queue
    grants = load_grants().set_index("grant_id")
    print(f"Layer 3: {len(queue)} grants to analyze"
          + (f"; {len(overflow)} more routed to deep review but over the cap (logged)" if len(overflow) else ""))

    def analyze(l2row: pd.Series) -> tuple[dict, list[dict]]:
        gid = l2row["grant_id"]
        grant = grants.loc[gid]
        if isinstance(grant, pd.DataFrame): # duplicate id; Layer 0 should have removed it
            grant = grant.iloc[0]
        texts = grant_texts(grant)
        messages = [{"role": "system", "content": system.format(**blocks)},
                    {"role": "user", "content": user_tmpl.format(**blocks, **texts)}]
        key = request_hash(client.model, version, messages)
        cached = cache.get(gid, key)
        log = []
        if cached is not None:
            out, attempts = cached["parsed"], cached["attempts"]
        else:
            out, attempts, problems = None, [], []
            for _ in range(2): # one retry, telling the model what was wrong
                raw = client.complete(messages if not problems else messages + [
                    {"role": "assistant", "content": attempts[-1]},
                    {"role": "user", "content": "Your JSON had these problems: " + "; ".join(problems)}])

```

```

        + ". Return the corrected JSON object only."}})

    attempts.append(raw)
    try:
        candidate = parse_json(raw)
        problems = validate(candidate, agency)
    except (ValueError, json.JSONDecodeError) as e:
        candidate, problems = None, [f"unparseable JSON: {e}"]
    if not problems:
        out = candidate
        break
    if len(attempts) > 1:
        log.append({"check": "invalid_output_retried", "severity": "medium",
                    "detail": f"first answer rejected by validation; {'fixed on retry' if out else 'still"}
invalid: ' + ' '; '.join(problems)}")
        cache.put(gid, key, client.model, version, {"parsed": out, "attempts": attempts})
    if out is None:
        log.append({"check": "invalid_output", "severity": "high", "detail": "no valid JSON after retry"})
        return {"grant_id": gid, "status": "invalid_output"}, log
    checks, errs = verify(out, texts)
    log += errs + compare_layers(l2row, out)
    row = {"grant_id": gid, "opportunity_title": l2row["opportunity_title"], "track": l2row["track"],
           "status": "ok", **{k: out.get(k) for k in (
               "strategic_alignment", "matched_goal", "matched_objective", "strategic_reason",
               "strategic_evidence_quote", "applicant_role", "eligibility_evidence_quote",
               "owning_division", "operational_fit", "operational_reason", "major_risk",
               "recommended_for_review", "one_line_rationale")}},
           "restrictions": json.dumps(out.get("restrictions", [])), **checks,
           "model": client.model, "prompt_version": version, "from_cache": cached is not None}
    return row, log

rows, error_log = [], []
with ThreadPoolExecutor(max_workers=workers) as pool:
    futures = {pool.submit(analyze, r): r for _, r in queue.iterrows()}
    for i, fut in enumerate(futures, 1):
        l2row = futures[fut]
        try:
            row, log = fut.result()
        except Exception as e:
            row, log = {"grant_id": l2row["grant_id"], "status": "api_error"}, [
                {"check": "api_error", "severity": "high", "detail": str(e)[:300]}]
        rows.append(row)
        error_log += [{"grant_id": l2row["grant_id"], "opportunity_title": l2row["opportunity_title"],
                       "layer": 3, **e} for e in log]
        if i % 25 == 0:
            print(f" {i}/{len(futures)}")
    for _, r in overflow.iterrows():
        error_log.append({"grant_id": r["grant_id"], "opportunity_title": r["opportunity_title"], "layer": 3,
                         "check": "over_layer3_cap", "severity": "info",
                         "detail": f"routed to deep review but beyond layer3.max_grants={cap}; not analyzed"})

result = pd.DataFrame(rows)
result.to_csv(out_dir / "deep_analysis.csv", index=False)
log_df = pd.DataFrame(error_log, columns=["grant_id", "opportunity_title", "layer", "check", "severity",
"detail"])
log_df.to_csv(out_dir / "ai_error_log.csv", index=False)
print(result["status"].value_counts().to_string() if len(result) else "no grants analyzed")
if len(log_df):
    print("AI error log:\n" + log_df["check"].value_counts().to_string())
print(f"Wrote {out_dir / 'deep_analysis.csv'} and {out_dir / 'ai_error_log.csv'}"
      + (" [MOCK - fake answers]" if mock else ""))
return result

if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--mock", action="store_true", help="fake model answers; no API key needed")
    ap.add_argument("--limit", type=int, help="analyze at most N grants (overrides config cap)")
    ap.add_argument("--workers", type=int, default=4)
    a = ap.parse_args()
    run(mock=a.mock, limit=a.limit, workers=a.workers)

```

src/scoring.py

```
"""Layer 4 - transparent, deterministic scoring for NCDHHS grants.

Layer 3 supplies the semantic judgments that require reading:
* strategic_alignment (0-10)
* operational_fit (0-10)
* verified_applicant-role / evidence fields

Layer 1 supplies objective scores:
* financial_value_score (0-10)
* deadline_score (0-10, nullable when the deadline is genuinely unknown)

Python combines those signals into the final ranking. No model ranks grants.

Outputs:
    data/results/ranked_grants.csv    open opportunities
    data/results/watchlist.csv       forecast opportunities
"""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import pandas as pd

import config

DEFAULT_WEIGHTS = {
    "strategic_alignment": 0.30,
    "operational_fit": 0.25,
    "financial_value_score": 0.15,
    "deadline_score": 0.15,
    "eligibility_confidence_score": 0.15,
}

DEFAULT_ROLE_SCORES = {
    # Eligibility confidence answers "can NCDHHS participate?", not
    # "is NCDHHS the most typical lead applicant?". Operational fit captures
    # the latter concern, so all verified eligible roles receive full credit.
    "lead": 10.0,
    "partner": 10.0,
    "atypical": 10.0,
    "unclear": 5.0,
    "ineligible": 0.0,
}

DEFAULT_MISSING_DEADLINE_PENALTY = 3.0
# A rolling ("accepted anytime") grant has a known, flexible deadline rather than an
# unknown one: a smaller penalty, so a similar grant with a real deadline comes first.
# If the money can run out (first come, first served) there is no flexibility, so no penalty.
DEFAULT_ROLLING_DEADLINE_PENALTY = 2.0
DEFAULT_ROLLING_FUNDS_LIMITED_PENALTY = 0.0
DEFAULT_UNVERIFIED_STRATEGIC_PENALTY = 5.0

def missing_deadline_penalties(agency: dict) -> dict[str, float]:
    """Penalty points for an open grant with no deadline score, by kind of missing deadline."""
    cfg = agency.get("layer4_scoring", {})
    out = {
        "unknown": float(cfg.get("missing_open_deadline_penalty_points", DEFAULT_MISSING_DEADLINE_PENALTY)),
        "rolling": float(cfg.get("rolling_deadline_penalty_points", DEFAULT_ROLLING_DEADLINE_PENALTY)),
        "rolling_funds_limited": float(cfg.get("rolling_funds_limited_penalty_points",
                                                DEFAULT_ROLLING_FUNDS_LIMITED_PENALTY)),
    }
```

```

    if any(v < 0 for v in out.values()):
        raise ValueError("Layer 4 penalties cannot be negative")
    return out

def scoring_config(agency: dict) -> tuple[dict[str, float], dict[str, float], float, float]:
    """Load and validate Layer 4 settings from priorities.json."""
    cfg = agency.get("layer4_scoring", {})
    weights = {**DEFAULT_WEIGHTS, **cfg.get("weights", {})}
    role_scores = {**DEFAULT_ROLE_SCORES, **cfg.get("eligibility_confidence_by_verified_role", {})}
    deadline_penalty = float(
        cfg.get("missing_open_deadline_penalty_points", DEFAULT_MISSING_DEADLINE_PENALTY)
    )
    evidence_penalty = float(
        cfg.get("unverified_strategic_evidence_penalty_points", DEFAULT_UNVERIFIED_STRATEGIC_PENALTY)
    )

    if abs(sum(weights.values()) - 1.0) > 1e-9:
        raise ValueError(f"Layer 4 weights must sum to 1.0; got {sum(weights.values()):.6f}")
    if any(w < 0 for w in weights.values()):
        raise ValueError("Layer 4 weights cannot be negative")
    if any(not 0 <= v <= 10 for v in role_scores.values()):
        raise ValueError("Eligibility-confidence role scores must be between 0 and 10")
    if deadline_penalty < 0 or evidence_penalty < 0:
        raise ValueError("Layer 4 penalties cannot be negative")

    return weights, role_scores, deadline_penalty, evidence_penalty

def _number(row: pd.Series, key: str) -> float | None:
    value = row.get(key)
    if value is None or pd.isna(value):
        return None
    try:
        return float(value)
    except (TypeError, ValueError):
        return None

def _bool(value) -> bool | None:
    if value is None or pd.isna(value):
        return None
    if isinstance(value, bool):
        return value
    text = str(value).strip().lower()
    if text in {"true", "1", "yes"}:
        return True
    if text in {"false", "0", "no"}:
        return False
    return None

def verified_role(row: pd.Series) -> str:
    """Use Layer 3's evidence-checked role; fall back to unclear."""
    role = row.get("applicant_role_verified")
    if isinstance(role, str) and role:
        return role
    return "unclear"

def confirmed_ineligible(row: pd.Series) -> bool:
    """Hard-gate only evidence-verified ineligibility.

    A model saying "ineligible" without a verified eligibility quote is not
    enough to remove a grant from consideration.
    """
    return (
        verified_role(row) == "ineligible"
        and _bool(row.get("eligibility_quote_verified")) is True
    )

```

```

def eligibility_confidence(row: pd.Series, role_scores: dict[str, float]) -> float:
    """Convert verified eligibility evidence to the 0-10 scoring component."""
    role = verified_role(row)

    # If eligibility evidence was not verified, treat the result as uncertain
    # rather than trusting a confident-sounding model label.
    if _bool(row.get("eligibility_quote_verified")) is not True:
        return role_scores.get("unclear", 5.0)

    return float(role_scores.get(role, role_scores.get("unclear", 5.0)))

def _check_0_10(name: str, value: float | None) -> None:
    if value is None:
        raise ValueError(f"{name} is required for Layer 4 scoring")
    if not 0 <= value <= 10:
        raise ValueError(f"{name} must be between 0 and 10; got {value}")

def score_grant(
    row: pd.Series,
    agency: dict | None = None,
) -> dict:
    """Score one grant and return an auditable scoring breakdown.

    Final scores are on a 0-100 scale. For forecasts, deadline is intentionally
    excluded and the remaining weights are re-normalized. For an open grant
    whose deadline is missing from the dataset, deadline is also excluded, but
    a small uncertainty penalty is applied.
    """
    agency = agency or config.load_agency()
    weights, role_scores, missing_deadline_penalty, evidence_penalty = scoring_config(agency)

    is_forecast = _bool(row.get("is_forecast"))
    if is_forecast is None:
        is_forecast = str(row.get("track", "")).strip().lower() == "forecast"
    track = "forecast" if is_forecast else "open"

    strategic = _number(row, "strategic_alignment")
    operational = _number(row, "operational_fit")
    financial = _number(row, "financial_value_score")
    deadline = _number(row, "deadline_score")

    _check_0_10("strategic_alignment", strategic)
    _check_0_10("operational_fit", operational)
    _check_0_10("financial_value_score", financial)
    if deadline is not None:
        _check_0_10("deadline_score", deadline)

    elig = eligibility_confidence(row, role_scores)

    if confirmed_ineligible(row):
        return {
            "track": track,
            "eligibility_confidence_score": 0.0,
            "deadline_weight_used": 0.0 if is_forecast else weights["deadline_score"],
            "weight_used": 1.0 - weights["deadline_score"] if is_forecast else 1.0,
            "missing_deadline_penalty": 0.0,
            "strategic_evidence_penalty": 0.0,
            "final_score": 0.0,
            "score_status": "confirmed_ineligible",
        }

    components = {
        "strategic_alignment": strategic,
        "operational_fit": operational,
        "financial_value_score": financial,
        "eligibility_confidence_score": elig,
    }

```

```

}

weighted_sum = sum(components[k] * weights[k] for k in components)
weight_used = sum(weights[k] for k in components)
deadline_weight_used = 0.0
missing_deadline = deadline is None

if not is_forecast and deadline is not None:
    weighted_sum += deadline * weights["deadline_score"]
    weight_used += weights["deadline_score"]
    deadline_weight_used = weights["deadline_score"]

# Forecasts intentionally omit deadline. Open grants with a genuinely
# missing deadline are re-normalized over the four known criteria.
base_10 = weighted_sum / weight_used
final_100 = base_10 * 10.0

applied_deadline_penalty = 0.0
deadline_kind = None
if not is_forecast and missing_deadline:
    if _bool(row.get("rolling_funds_limited")) is True:
        deadline_kind = "rolling_funds_limited"
    elif _bool(row.get("rolling_deadline")) is True:
        deadline_kind = "rolling"
    else:
        deadline_kind = "unknown"
    applied_deadline_penalty = missing_deadline_penalties(agency)[deadline_kind]
    final_100 -= applied_deadline_penalty

# Existing Layer 3 design requires unsupported strategic evidence to hurt
# the ranking rather than silently trusting the model's 0-10 judgment.
applied_evidence_penalty = 0.0
if _bool(row.get("strategic_quote_verified")) is False:
    applied_evidence_penalty = evidence_penalty
    final_100 -= applied_evidence_penalty

final_100 = round(max(0.0, min(100.0, final_100)), 2)

if is_forecast:
    status = "forecast_watchlist"
elif deadline_kind == "rolling_funds_limited":
    status = "open_rolling_funds_limited"
elif deadline_kind == "rolling":
    status = "open_rolling"
elif missing_deadline:
    status = "open_missing_deadline"
else:
    status = "open"

return {
    "track": track,
    "eligibility_confidence_score": round(elig, 2),
    "deadline_weight_used": round(deadline_weight_used, 4),
    "weight_used": round(weight_used, 4),
    "missing_deadline_penalty": round(applied_deadline_penalty, 2),
    "strategic_evidence_penalty": round(applied_evidence_penalty, 2),
    "final_score": final_100,
    "score_status": status,
}

```

```

def score_dataframe(df: pd.DataFrame, agency: dict | None = None) -> pd.DataFrame:
    """Score every row, preserving the inputs for explainability."""
    agency = agency or config.load_agency()
    scored = df.copy()
    breakdowns = scored.apply(lambda r: pd.Series(score_grant(r, agency)), axis=1)
    for column in breakdowns.columns:
        scored[column] = breakdowns[column]
    return scored

```

```

def rank_track(df: pd.DataFrame) -> pd.DataFrame:
    """Rank one track with deterministic tie-breakers."""
    ranked = df.sort_values(
        ["final_score", "strategic_alignment", "operational_fit", "grant_id"],
        ascending=[False, False, False, True],
        kind="stable",
    ).copy()
    ranked.insert(0, "rank", range(1, len(ranked) + 1))
    return ranked

def load_inputs(out_dir: Path) -> pd.DataFrame:
    """Join Layer 1 objective features to successful Layer 3 analyses."""
    if not config.LAYER1_OUTPUT.exists():
        raise FileNotFoundError(
            f"{config.LAYER1_OUTPUT} does not exist. Layer 4 requires the real "
            "Layer 1 output with financial_value_score and deadline_score."
        )

    deep_path = out_dir / "deep_analysis.csv"
    if not deep_path.exists():
        raise FileNotFoundError(f"{deep_path} does not exist; run Layer 3 first")

    features = pd.read_csv(config.LAYER1_OUTPUT)
    deep = pd.read_csv(deep_path)
    if "status" in deep.columns:
        failed = deep[deep["status"] != "ok"]
        if len(failed):
            print(f"WARNING: {len(failed)} grants have no valid Layer 3 analysis "
                  f"({failed['status'].value_counts().to_dict()}) and are NOT ranked. "
                  f"See ai_error_log.csv; rerun deep_analysis.py to retry.")
        deep = deep[deep["status"] == "ok"].copy()

    required_features = {"grant_id", "is_forecast", "financial_value_score", "deadline_score"}
    missing = required_features - set(features.columns)
    if missing:
        raise ValueError(
            "Layer 1 output is missing fields required by Layer 4: "
            + ", ".join(sorted(missing))
        )

    required_deep = {
        "grant_id",
        "strategic_alignment",
        "operational_fit",
        "applicant_role_verified",
        "eligibility_quote_verified",
        "strategic_quote_verified",
    }
    missing = required_deep - set(deep.columns)
    if missing:
        raise ValueError(
            "Layer 3 output is missing fields required by Layer 4: "
            + ", ".join(sorted(missing))
        )

    # Keep Layer 1 as the source of objective/raw fields and Layer 3 as the
    # source of semantic judgments. Overlapping descriptive fields from Layer 3
    # receive a _layer3 suffix rather than silently overwriting Layer 1.
    return features.merge(deep, on="grant_id", how="inner", suffixes=("", "_layer3"))

def split_research(scored: pd.DataFrame) -> pd.Series:
    """True for research/training mechanisms (from Layer 1); missing column -> none."""
    if "research_mechanism" not in scored.columns:
        return pd.Series(False, index=scored.index)
    return scored["research_mechanism"].map(lambda v: _bool(v) is True)

```

```

def run(mock: bool = False) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Create the open ranking and forecast watchlist CSVs."""
    agency = config.load_agency()
    out_dir = config.MOCK_RESULTS if mock else config.RESULTS
    out_dir.mkdir(parents=True, exist_ok=True)

    joined = load_inputs(out_dir)
    scored = score_dataframe(joined, agency)

    # Research/training mechanisms (Layer 1 `research_mechanism`: NIH, or an activity code such
    # as R01/K99/U01 in the title) are scored the same way but listed separately: a state DHHS
    # cannot realistically lead them, though it may want to pass them to a university partner.
    research = split_research(scored)
    open_ranked = rank_track(scored[(scored["track"] == "open") & ~research])
    watchlist = rank_track(scored[(scored["track"] == "forecast") & ~research])
    research_ranked = rank_track(scored[research])

    paths = {"ranked_grants.csv": open_ranked, "watchlist.csv": watchlist,
            "research_partnerships.csv": research_ranked}
    for name, frame in paths.items():
        frame.to_csv(out_dir / name, index=False)
        print(f"Wrote {out_dir / name} ({len(frame)} grants)")
    return open_ranked, watchlist

if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--mock", action="store_true", help="read mock Layer 3 output")
    args = ap.parse_args()
    run(mock=args.mock)

```

src/award_history.py

"""Layer 5 - historical-award check with USAspending.gov (objective, no AI).

For every grant's assistance listings (ALNs, e.g. 93.917), ask USAspending who actually received grant awards under those listings in FY2022-FY2025:

- * did NC DHHS receive any? (recipient name match, NC recipients)
- * did state agencies receive a real share?
- * or did the money go mainly to universities, tribes, local governments, or nonprofits/hospitals/companies?

This answers "is this a program state agencies like DHHS actually win". It is NOT ground truth for strategic fit, and an ALN covers every NOFO under it, not only this one.

Recipients are classified by NAME RULES (below), not USAspending's recipient-type tags, which are incomplete: for ALN 93.917 NC DHHS received \$187.6M by name but only \$91.2M is tagged as state government. Names the rules cannot place are reported as `unclassified`; only the top recipients are fetched, so each result carries a `coverage` share and is marked `unclear` when coverage is too low.

Every API response is cached in data/cache/usaspending/, so reruns are offline.

Usage:

```
python src/award_history.py          # all grants Layer 2 triaged (needed for the L2 comparison)
python src/award_history.py --offline # cache only; fail on anything not cached
```

"""

```
import argparse
import hashlib
import json
import re
import time
from concurrent.futures import ThreadPoolExecutor
```

```
import pandas as pd
import requests
```

```
import config
```

```
API = "https://api.usaspending.gov/api/v2"
CACHE = config.CACHE / "usaspending"
OUT_GRANTS = config.RESULTS / "award_history.csv"
OUT_ALNS = config.RESULTS / "award_history_alns.csv"
REPORT = config.RESULTS / "award_history_report.md"
ALN_RE = re.compile(r"^\d{2}\.\d{3}$")
```

```
STATES = ["ALABAMA", "ALASKA", "ARIZONA", "ARKANSAS", "CALIFORNIA", "COLORADO", "CONNECTICUT", "DELAWARE",
"FLORIDA", "GEORGIA", "HAWAII", "IDAHO", "ILLINOIS", "INDIANA", "IOWA", "KANSAS", "KENTUCKY",
"LOUISIANA", "MAINE", "MARYLAND", "MASSACHUSETTS", "MICHIGAN", "MINNESOTA", "MISSISSIPPI", "MISSOURI",
"MONTANA", "NEBRASKA", "NEVADA", "NEW HAMPSHIRE", "NEW JERSEY", "NEW MEXICO", "NEW YORK",
"NORTH CAROLINA", "NORTH DAKOTA", "OHIO", "OKLAHOMA", "OREGON", "PENNSYLVANIA", "RHODE ISLAND",
"SOUTH CAROLINA", "SOUTH DAKOTA", "TENNESSEE", "TEXAS", "UTAH", "VERMONT", "VIRGINIA", "WASHINGTON",
"WEST VIRGINIA", "WISCONSIN", "WYOMING", "DISTRICT OF COLUMBIA", "PUERTO RICO", "GUAM",
"VIRGIN ISLANDS", "AMERICAN SAMOA", "NORTHERN MARIANA"]
```

```
_STATE_NAME = re.compile(r"\b(" + "|".join(STATES) + r")\b")
```

```
_UNIVERSITY = re.compile(r"UNIVERSITY|UNIV|COLLEGE|INSTITUTE OF TECHNOLOGY|POLYTECHNIC|REGENTS|BOARD OF TRUSTEES|"
```

```
    r"SCHOOL OF MEDICINE|MEDICAL SCHOOL|HEALTH SCIENCE CENTER|RESEARCH FOUNDATION")
```

```
_TRIBAL = re.compile(r"\bTRIBES?\b|\bTRIBAL\b|PUEBLO|RANCHERIA|\bBAND OF\b|\bNATION\b|INDIAN|NATIVE VILLAGE|"
    r"ALASKA NATIVE|NATIVE AMERICAN|NATIVE HAWAIIAN")
```

```
_LOCAL = re.compile(r"\bCOUNTY\b|\bCITY\b|\bTOWN( OF)?\b|VILLAGE OF|BOROUGH|PARISH|MUNICIPAL|SCHOOL DISTRICT|"
    r"PUBLIC SCHOOLS|\bDISTRICT HEALTH\b")
```

```
_GOV_WORD = re.compile(r"DEPARTMENT|\bDEPT\b|DIVISION|OFFICE OF|AGENCY|EXECUTIVE|BUREAU|COMMISSION|AUTHORITY|"
    r"ADMINISTRATION|CABINET|SECRETARY|GOVERNOR|STATE OF|COMMONWEALTH|\bSTATE\b")
```

```
_NOT_REAL = re.compile(r"MULTIPLE RECIPIENTS|REDACTED|MISCELLANEOUS|^\$")
```

```
NC_DHHS = re.compile(r"NORTH CAROLINA DE(PARTMEN)?T\.? OF HEALTH|HEALTH (AND|&) HUMAN SERVICES,? NORTH CAROLINA|"
```

```

r"\bN\.\?C\.\? DEP(ARTMEN)?T\.\? OF HEALTH")
CLASSES = ["state_agency", "university", "tribal", "local_government", "nonprofit_or_other", "unclassified"]

def classify(name: str) -> str:
    """Deterministic recipient class from the recipient name. Order matters."""
    n = re.sub(r"\s+", " ", str(name or "")).upper().strip()
    if _NOT_REAL.search(n):
        return "unclassified"
    if _UNIVERSITY.search(n):
        return "university"
    if _TRIBAL.search(n):
        return "tribal"
    if _LOCAL.search(n):
        return "local_government"
    if (_STATE_NAME.search(n) or "STATE OF" in n or "COMMONWEALTH" in n) and _GOV_WORD.search(n):
        return "state_agency"
    return "nonprofit_or_other"

def parse_alns(listings) -> list[str]:
    if not isinstance(listings, str):
        return []
    return sorted({x.split("|")[0].strip() for x in listings.split(";") if x.strip()})

# — Cached API access —————

class Client:
    def __init__(self, offline: bool = False):
        self.offline = offline
        CACHE.mkdir(parents=True, exist_ok=True)
        self.session = requests.Session()

    def post(self, endpoint: str, payload: dict) -> dict:
        key = hashlib.sha256(json.dumps({"e": endpoint, "p": payload}, sort_keys=True).encode()).hexdigest()[:24]
        path = CACHE / f"{key}.json"
        if path.exists():
            return json.loads(path.read_text(encoding="utf-8"))["response"]
        if self.offline:
            raise RuntimeError(f"not cached (offline): {endpoint} {payload.get('filters', {})}")
        for attempt in range(5):
            try:
                r = self.session.post(f"API:{endpoint}", json=payload, timeout=90)
            except requests.RequestException:
                time.sleep(2 ** attempt)
                continue
            if r.status_code == 200:
                data = r.json()
                path.write_text(json.dumps({"endpoint": endpoint, "payload": payload, "response": data}),
                                encoding="utf-8")
                return data
            if r.status_code in (429, 500, 502, 503, 504):
                time.sleep(2 ** attempt)
                continue
            raise RuntimeError(f"HTTP {r.status_code} {endpoint}: {r.text[:150]}")
        raise RuntimeError(f"USAspending unreachable after retries: {endpoint}")

def aln_history(client: Client, aln: str, cfg: dict) -> dict:
    """Award history for one assistance listing."""
    filters = {"program_numbers": [aln], "award_type_codes": cfg["award_type_codes"],
               "time_period": [{"start_date": cfg["start_date"], "end_date": cfg["end_date"]}]}
    out = {"aln": aln, "total": 0.0, **{c: 0.0 for c in CLASSES}, "nc_dhhs": 0.0, "nc_state_agency": 0.0,
           "nc_state_agency_names": "", "top_recipients": "", "error": ""}
    if not ALN_RE.match(aln):
        out["error"] = "not a standard assistance listing number"
        return out
    try:

```

```

sot = client.post("/search/spending_over_time/", {"group": "fiscal_year", "filters": filters})
out["total"] = float(sum(x["aggregated_amount"] or 0 for x in sot["results"]))
if out["total"] <= 0:
    return out
recips = []
for page in range(1, cfg["national_recipient_pages"] + 1):
    res = client.post("/search/spending_by_category/recipient/",
                      {"filters": filters, "limit": 100, "page": page})
    recips += res.get("results", [])
    if not res.get("page_metadata", {}).get("hasNext"):
        break
for r in recips:
    out[classify(r.get("name"))] += float(r.get("amount") or 0)
out["top_recipients"] = "; ".join(f"{r.get('name')} ({(r.get('amount') or 0) / 1e6:.1f}M)" for r in
recips[:3])
nc = client.post("/search/spending_by_category/recipient/",
                  {"filters": {"**filters, "recipient_locations": [{"country": "USA", "state": "NC"}]},
                  "limit": 100, "page": 1})
nc_state = []
for r in nc.get("results", []):
    name, amt = str(r.get("name") or ""), float(r.get("amount") or 0)
    if NC_DHHS.search(name.upper()):
        out["nc_dhhs"] += amt
    if classify(name) == "state_agency":
        out["nc_state_agency"] += amt
        nc_state.append(name)
out["nc_state_agency_names"] = "; ".join(sorted(set(nc_state)))
except RuntimeError as e:
    out["error"] = str(e)[:200]
return out

# — Per-grant verdicts —————

def verdict(row: pd.Series, cfg: dict) -> str:
    if row["error"] and row["total"] == 0:
        return "unknown (API error or non-standard listing)"
    if row["total"] <= 0:
        return f"no awards found {cfg['fiscal_years_label']}"
    if row["nc_dhhs"] >= cfg["nc_dhhs_min_usd"]:
        return "NC DHHS has received awards"
    share = lambda c: row[c] / row["total"]
    if share("state_agency") >= cfg["state_agency_share_threshold"]:
        return "other state agencies win this"
    classified = sum(row[c] for c in CLASSES if c != "unclassified") / row["total"]
    if classified < cfg["min_classified_coverage"]:
        return "unclear (top recipients cover too little of the money)"
    for c, label in (("university", "mostly universities"), ("tribal", "mostly tribal / Native organizations"),
                     ("local_government", "mostly local governments"),
                     ("nonprofit_or_other", "mostly nonprofits, hospitals or companies")):
        if share(c) >= cfg["majority_share"]:
            return label
    return "mixed recipients" + (" (some state agencies)" if row["state_agency"] > 0 else " (no state agencies)")

def grant_history(grants: pd.DataFrame, alns: pd.DataFrame, cfg: dict) -> pd.DataFrame:
    by_aln = alns.set_index("aln")
    rows = []
    for g in grants.itertuples():
        codes = [a for a in g.alns if a in by_aln.index]
        sub = by_aln.loc[codes] if codes else by_aln.iloc[0:0]
        rec = {"grant_id": g.grant_id, "opportunity_title": g.opportunity_title, "alns": ";".join(g.alns),
               **{k: float(sub[k].sum()) if len(sub) else 0.0
                  for k in ["total", *CLASSES, "nc_dhhs", "nc_state_agency"]},
               "nc_state_agency_names": "; ".join(sorted({n for s in sub["nc_state_agency_names"] for n in s.split(";")
               if n})),
               "top_recipients": " | ".join(s for s in sub["top_recipients"] if s)[:400],
               "error": "; ".join(sorted({e for e in sub["error"] if e})):200] if len(sub) else "no assistance
listing"}
        rec["state_agency_share"] = rec["state_agency"] / rec["total"] if rec["total"] else 0.0

```

```

rec["university_share"] = rec["university"] / rec["total"] if rec["total"] else 0.0
rec["coverage"] = sum(rec[c] for c in CLASSES if c != "unclassified") / rec["total"] if rec["total"] else 0.0
rec["verdict"] = verdict(pd.Series(rec), cfg)
rec["state_agency_history"] = (rec["nc_dhhs"] >= cfg["nc_dhhs_min_usd"]
                              or rec["state_agency_share"] >= cfg["state_agency_share_threshold"])

rows.append(rec)
return pd.DataFrame(rows)

# — Report —————

def _m(x) -> str:
    return f"${x / 1e6:,.1f}M" if abs(x) >= 1e6 else f"${x / 1e3:,.0f}K"

def _pct(x) -> str:
    # De-obligations (negative amounts) can push a share slightly outside 0-100%.
    return f"{max(0.0, min(1.0, x)) + 0.0:.0%}"

def _broad(alns: str) -> str:
    n = len([a for a in str(alns).split(";") if a])
    return f" (spans {n} listings)" if n > 3 else ""

def report(hist: pd.DataFrame, cfg: dict) -> list[str]:
    jev = pd.read_csv(config.RESULTS / "jev_outputs.csv")[["grant_id", "route", "domain_relevance",
"applicant_role"]]
    deep = pd.read_csv(config.RESULTS / "deep_analysis.csv")
    h = hist.merge(jev, on="grant_id", how="left")
    fy = cfg["fiscal_years_label"]
    L = [f"# Historical-award check (USAspending.gov)", "",
        f"Objective, non-AI check: who actually received grant awards under each grant's assistance listings in "
        f"**{fy}** (USAspending.gov, award types: block, formula and project grants, cooperative agreements). "
        f"It answers *\"is this a program state agencies like DHHS actually win?\"*. It does **not** measure "
        f"strategic fit, and an assistance listing covers every funding notice under it, not only this one.", "",
        f"**How recipients are classified:** by name rules in `src/award_history.py` (state agency / university / "
        f"tribal / local government / nonprofit-hospital-company), because USAspending's own recipient-type tags are "
        f"
        f"incomplete (for listing 93.917, NC DHHS received $187.6M by name but only $91.2M is tagged as state "
        f"government). Only the top 200 recipients per listing are fetched; `coverage` is the share of dollars those "
        f"
        f"cover. 'State agency history' = NC DHHS received at least "
        f"f"_{m(cfg['nc_dhhs_min_usd'])}, or state agencies received at least "
        f"f"_{cfg['state_agency_share_threshold']:.0%} of the dollars.", "",
        f"Grants checked: **{len(h)}** (every grant Jev triaged). Distinct assistance listings queried: "
        f"f"_{h['alns'].str.split(';').explode().replace('', pd.NA).dropna().nunique()}**.", ""]

    def show(df, title):
        rows = [f"### {title}", "", "| # | Grant | Verdict | NC DHHS | State-agency share | University share | "
        Coverage |",
            " |---|---|---|---|---|---|"]
        for i, r in enumerate(df.iteruples(), 1):
            rows.append(f"| {i} | {r.opportunity_title[:70]} | {r.verdict}_{broad(r.alns)} | "
                f"f"_{m(r.nc_dhhs)} if r.nc_dhhs > 0 else '-' | "
                f"f"_{pct(r.state_agency_share)} | {pct(r.university_share)} | {pct(r.coverage)} |")
        n = len(df)
        rows += [f"", f"**{int((df['nc_dhhs'] >= cfg['nc_dhhs_min_usd']).sum())} of {n}** have awards to NC DHHS "
            f"f"(at least {m(cfg['nc_dhhs_min_usd'])} in {fy}); "
            f"f"**{int(df['state_agency_history'].sum())} of {n}** have NC DHHS or substantial state-agency "
            f"f"awards; {int(df['verdict'].str.startswith('no awards').sum())} have no awards in {fy}; "
            f"f"_{int(df['verdict'].str.startswith(('unknown', 'unclear')).sum())} unknown/unclear.", ""]
        return rows

    top_open = pd.read_csv(config.RESULTS / "ranked_grants.csv").head(10)[["grant_id"]]
    top_watch = pd.read_csv(config.RESULTS / "watchlist.csv").head(5)[["grant_id"]]
    L += show(top_open.merge(h, on="grant_id"), "Top 10 open grants")
    L += show(top_watch.merge(h, on="grant_id"), "Top 5 watchlist grants")

    dep = h[h["route"] == "deprioritized"]

```

```

dr = h[h["route"] == "deep_review"]
fn = dep[dep["state_agency_history"]]
uni_only = dr[(dr["verdict"] == "mostly universities") & (dr["state_agency"] == 0) & (dr["nc_dhhs"] == 0)]
L += ["## Compared with Layer 2 (Jev) routing", "",
      f"- **Deprioritized by Jev:** {len(dep)}. Of these, **{len(fn)** have a history of awards to NC DHHS or "
      f"substantial state-agency awards (possible false negatives). "
      f"{int((dep['nc_dhhs'] >= cfg['nc_dhhs_min_usd']).sum())} of them under a listing where NC DHHS itself
received awards.",
      f"- **Sent to deep review:** {len(dr)}. Of these, **{len(uni_only)** went mostly to universities with no "
      f"state-agency recipients among the top 200 (probably research-type programs).",
      f"- Grants Jev deprioritized vs sent on, share with state-agency history: "
      f"{dep['state_agency_history'].mean():.0%} vs {dr['state_agency_history'].mean():.0%}.", ""]
if len(fn):
    L += ["### Deprioritized grants with state-agency award history (inspect each)", "",
          "| Grant | Jev relevance / role | Verdict | NC DHHS | State share |", "|---|---|---|---|---|"]
    for r in fn.sort_values(["nc_dhhs", "state_agency_share"], ascending=False).head(25).itertuples():
        L.append(f"| {r.opportunity_title[:90]} | {r.domain_relevance} / {r.applicant_role} | {r.verdict}
{broad(r.alns)} | "
                f"{_m(r.nc_dhhs) if r.nc_dhhs > 0 else '-'} | {_pct(r.state_agency_share)} |")
    L += ["", "_Caveat: an assistance listing is shared by many funding notices. A listing states win (e.g. a "
          "broad CDC or HRSA listing) can still carry a specific notice meant for others, such as a "
          "tribal-only or university-only competition. Treat these as leads to check, not proven misses. "
          "Our inspection of each is in `docs/failure_modes.md` (F9)._", ""]
L += ["## Verdicts across all checked grants", "",
      h["verdict"].value_counts().rename("grants").to_frame().to_markdown(), ""]
return L

```

```

def run(offline: bool = False, workers: int = 6) -> pd.DataFrame:
    agency = config.load_agency()
    cfg = agency["layer5_award_history"]
    feats = pd.read_csv(config.LAYER1_OUTPUT, dtype=str, keep_default_na=False)
    jev = pd.read_csv(config.RESULTS / "jev_outputs.csv", dtype=str)
    grants = feats[feats["grant_id"].isin(jev["grant_id"])]
    grants["alns"] = grants["opportunity_assistance_listings"].map(parse_alns)
    codes = sorted({a for l in grants["alns"] for a in l})
    client = Client(offline=offline)
    print(f"Award history: {len(grants)} grants, {len(codes)} assistance listings "
          f"({cfg['fiscal_years_label']}, cache: {CACHE})")
    with ThreadPoolExecutor(max_workers=workers) as pool:
        alns = pd.DataFrame(list(pool.map(lambda a: aln_history(client, a, cfg), codes)))
    alns.to_csv(OUT_ALNS, index=False)
    errs = alns[alns["error"] != ""]
    if len(errs):
        print(f" {len(errs)} listings could not be answered: {errs['error'].str[:60].value_counts().to_dict()}")
    hist = grant_history(grants, alns, cfg)
    hist.to_csv(OUT_GRANTS, index=False)
    lines = report(hist, cfg)
    REPORT.write_text("\n".join(lines) + "\n", encoding="utf-8")
    print("\n".join(lines[:4]))
    print(f"Wrote {OUT_GRANTS}, {OUT_ALNS}, {REPORT}")
    return hist

```

```

if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--offline", action="store_true", help="use cached responses only")
    ap.add_argument("--workers", type=int, default=6)
    a = ap.parse_args()
    run(offline=a.offline, workers=a.workers)

```

src/validation.py

```
"""Layer 5 - validation. Human labels decide whether the pipeline works.
```

```
Three steps:
```

1. `python src/validation.py sample`
Draws a STRATIFIED sample of grants that reach Layer 2 and writes a blind labeling sheet: `data/validation/labels.csv`. Strata come from the keyword baseline (not from Jev), so the sample does not depend on the model being evaluated and can be labeled before any API run. Rows are shuffled and the sheet shows no model output or stratum. Two teammates label every row.
2. `python src/validation.py spotcheck` (after the real pipeline run)
Writes `data/validation/spotcheck.csv`: the top 10 open grants, the top 5 watchlist grants, 5 random grants ranked 50+, and 5 random grants Layer 2 deprioritized - to check each against our criteria by hand.
3. `python src/validation.py evaluate`
Reads the labels and the pipeline outputs and writes `data/results/validation_report.md`: labeler agreement, Layer 2 recall and precision with stratified-bootstrap confidence intervals, the keyword baseline at the same review budget, every false negative, and a summary of the AI error log.

```
Label meaning ("relevant"): the DHHS grants coordinator should spend time on this grant - DHHS could plausibly lead or be a named partner AND it advances a DHHS strategic goal. Deadline and award size are NOT part of the label.
"""
```

```
import argparse
import json
import sys
```

```
import numpy as np
import pandas as pd
```

```
import config
from keyword_baseline import OUTPUT as BASELINE
from text_utils import strip_html, truncate
```

```
VAL = config.DATA / "validation"
LABELS = VAL / "labels.csv"
DESIGN = VAL / "sample_design.csv"
SPOTCHECK = VAL / "spotcheck.csv"
REPORT = config.RESULTS / "validation_report.md"
SEED = 42
# Stratum -> (baseline rank range, sample size)
STRATA = {"S1_top100": (1, 100, 30), "S2_rank101_400": (101, 400, 30), "S3_rank401_plus": (401, 10**9, 30)}
LABEL_COLUMNS = ["labeler_A_relevant", "labeler_A_role", "labeler_B_relevant", "labeler_B_role",
                 "final_relevant", "notes"]
YES, NO = {"y", "yes", "1", "true"}, {"n", "no", "0", "false"}
```

```
def read_labels() -> pd.DataFrame:
    """Read the labeling sheet. Excel often re-saves CSVs in the Windows code page instead of UTF-8 (and may add a BOM), so try both rather than crash on a curly quote or en dash."""
    for enc in ("utf-8-sig", "cp1252"):
        try:
            return pd.read_csv(LABELS, dtype=str, encoding=enc).fillna("")
        except UnicodeDecodeError:
            continue
    return pd.read_csv(LABELS, dtype=str, encoding="latin-1").fillna("")
```

```
# — 1. Sampling —————
```

```
def draw_sample(features: pd.DataFrame, baseline: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
```

```

live = features[~features["hard_filtered"].astype(bool)].merge(
    baseline[["grant_id", "baseline_rank"]], on="grant_id")
live["baseline_rank"] = live["baseline_rank"].astype(int)
rng = np.random.default_rng(SEED)
picks, design = [], []
for name, (lo, hi, n) in STRATA.items():
    pool = live[live["baseline_rank"].between(lo, hi)]
    take = pool.iloc[rng.choice(len(pool), size=min(n, len(pool)), replace=False)]
    picks.append(take.assign(stratum=name))
    design.append({"stratum": name, "baseline_rank_from": lo, "baseline_rank_to": min(hi, len(live)),
                  "population_size": len(pool), "sample_size": len(take)})
sample = pd.concat(picks).sample(frac=1, random_state=SEED).reset_index(drop=True)
sample.insert(0, "sample_id", [f"V{i + 1:03d}" for i in range(len(sample))])
return sample, pd.DataFrame(design)

def write_sheet(sample: pd.DataFrame) -> pd.DataFrame:
    sheet = pd.DataFrame({
        "sample_id": sample["sample_id"],
        "grant_id": sample["grant_id"],
        "title": sample["opportunity_title"],
        "federal_agency": sample["agency_name"],
        "applicant_types": sample["applicant_types"].fillna("").str.replace(";", ", "),
        "eligibility_text": sample["applicant_eligibility_description"].fillna("").map(lambda t:
truncate(strip_html(t), 1500)),
        "summary": sample["summary_description"].fillna("").map(lambda t: truncate(strip_html(t), 2000)),
        "url": sample["url"],
    })
    for c in LABEL_COLUMNS:
        sheet[c] = ""
    return sheet

def cmd_sample(force: bool) -> None:
    if LABELS.exists() and not force:
        existing = read_labels()
        if (existing[[c for c in LABEL_COLUMNS if c in existing]] != "").any().any():
            sys.exit(f"{LABELS} already has labels in it. Refusing to overwrite human work "
                    f"(use --force only if you really mean to).")
    features = pd.read_csv(config.LAYER1_OUTPUT)
    if not BASELINE.exists():
        import keyword_baseline
        keyword_baseline.run()
    sample, design = draw_sample(features, pd.read_csv(BASELINE))
    VAL.mkdir(parents=True, exist_ok=True)
    write_sheet(sample).to_csv(LABELS, index=False)
    sample[["sample_id", "grant_id", "stratum"]].merge(design, on="stratum").to_csv(DESIGN, index=False)
    print(design.to_string(index=False))
    print(f"\nWrote blind labeling sheet {LABELS} ({len(sample)} grants).")
    print("Fill labeler_A_relevant / labeler_B_relevant with Y or N (and optionally the role: "
          "lead/partner/atypical/ineligible). Label independently; do not look at model output first.")

# — 2. Spot-check sheet —————

def cmd_spotcheck() -> None:
    ranked = pd.read_csv(config.RESULTS / "ranked_grants.csv")
    watch = pd.read_csv(config.RESULTS / "watchlist.csv")
    jev = pd.read_csv(config.RESULTS / "jev_outputs.csv")
    rng = np.random.default_rng(SEED)
    low = ranked[ranked["rank"] >= 50]
    depri = jev[jev["route"] == "deprioritized"]
    parts = [ranked.head(10).assign(group="top10_open"), watch.head(5).assign(group="top5_watchlist"),
             low.iloc[rng.choice(len(low), size=min(5, len(low)), replace=False)].assign(group="random_rank50plus"),
             depri.iloc[rng.choice(len(depri), size=min(5, len(depri))),
             replace=False]].assign(group="random_deprioritized")]
    cols = ["group", "rank", "grant_id", "opportunity_title", "final_score", "one_line_rationale"]
    sheet = pd.concat([p.reindex(columns=cols) for p in parts])
    for c in ["check_eligible", "check_aligned", "check_deadline_realistic", "check_award_worth_it",
              "verdict_should_be_here", "notes"]:

```

```

        sheet[c] = ""
    VAL.mkdir(parents=True, exist_ok=True)
    sheet.to_csv(SPOTCHECK, index=False)
    print(f"Wrote {SPOTCHECK} ({len(sheet)} rows). Fill the check_* columns Y/N against our 5 criteria.")

# — 3. Evaluation —————

def _yn(v) -> float:
    # Accepts "Y", "yes", "(Y)", "(N, comment)" - labelers annotate in the cell.
    v = str(v).strip().strip("(").split(",")[0].strip().lower()
    return 1.0 if v in YES else 0.0 if v in NO else np.nan

def resolve_labels(labels: pd.DataFrame) -> tuple[pd.DataFrame, list[str]]:
    a, b, final = (labels[c].map(_yn) for c in ("labeler_A_relevant", "labeler_B_relevant", "final_relevant"))
    # Two labels: they must agree (or be adjudicated in final_relevant). One label: use it, and the
    # report says it is single-labeler (agreement cannot be measured).
    single = a.where(b.isna(), b.where(a.isna()))
    resolved = final.where(final.notna(), a.where(a == b).where(a.notna() & b.notna(), single))
    labels = labels.assign(label_A=a, label_B=b, relevant=resolved,
                           single_labeler=(a.notna() ^ b.notna()) & final.isna())
    unresolved = labels.loc[a.notna() & b.notna() & (a != b) & final.isna(), "sample_id"].tolist()
    return labels, unresolved

def cohen_kappa(a: pd.Series, b: pd.Series) -> float:
    m = a.notna() & b.notna()
    a, b = a[m], b[m]
    if len(a) == 0:
        return float("nan")
    po = (a == b).mean()
    pe = a.mean() * b.mean() + (1 - a.mean()) * (1 - b.mean())
    return float("nan") if pe == 1 else (po - pe) / (1 - pe)

def weighted_rates(df: pd.DataFrame, retained_col: str) -> dict:
    """Design-weighted recall and precision. Each labeled grant stands for
    population_size / labeled_in_stratum grants of its stratum."""
    w = df["population_size"] / df.groupby("stratum")["relevant"].transform("count")
    rel, ret = df["relevant"] == 1, df[retained_col].astype(bool)
    tp = (w * (rel & ret)).sum()
    recall = tp / (w * rel).sum() if (w * rel).sum() else np.nan
    precision = tp / (w * ret).sum() if (w * ret).sum() else np.nan
    return {"recall": recall, "precision": precision}

def bootstrap_ci(df: pd.DataFrame, retained_col: str, reps: int = 2000) -> dict:
    rng = np.random.default_rng(SEED)
    groups = [g for _, g in df.groupby("stratum")]
    draws = {"recall": [], "precision": []}
    for _ in range(reps):
        boot = pd.concat([g.iloc[rng.integers(0, len(g), len(g))] for g in groups])
        r = weighted_rates(boot, retained_col)
        for k in draws:
            if not np.isnan(r[k]):
                draws[k].append(r[k])
    return {k: (np.percentile(v, 2.5), np.percentile(v, 97.5)) if v else (np.nan, np.nan)
            for k, v in draws.items()}

def fmt(x) -> str:
    return "n/a" if x is None or (isinstance(x, float) and np.isnan(x)) else f"{100 * x:.0f}%"

def cmd_evaluate() -> None:
    labels, unresolved = resolve_labels(read_labels())
    if unresolved:
        print(f"WARNING: labelers disagree on {len(unresolved)} grants with no final_relevant: "
              f"{', '.join(unresolved)}. They are excluded until adjudicated.")

```

```

design = pd.read_csv(DSIGN)
df = labels.merge(design[["sample_id", "stratum", "population_size"]], on="sample_id")
df = df[df["relevant"].notna()].copy()
if df.empty:
    sys.exit("No resolved labels yet - fill in labels.csv first.")

n_single = int(df["single_labeler"].sum())
lines = ["# Validation report", "",
        f"Labeled and resolved: **{len(df)}** grants "
        f"{len(unresolved)} awaiting adjudication). Label = 'the coordinator should spend time on this'.", ""]
if n_single:
    lines += [f"> **{n_single} of {len(df)} labels come from a single labeler** (a team member, not a grants "
            "expert), so labeler agreement cannot be measured and the labels themselves are uncertain. "
            "With this few labels, the intervals below are wide; read them as a first check, not a "
            "measurement.", ""]
kappa = cohen_kappa(labels["label_A"], labels["label_B"])
both = labels["label_A"].notna() & labels["label_B"].notna()
agree = (labels.loc[both, "label_A"] == labels.loc[both, "label_B"]).mean() if both.any() else np.nan
lines += ["## Do humans agree?", "",
        f"- Double-labeled: {int(both.sum())}; raw agreement {fmt(agree)}; Cohen's kappa **{kappa:.2f}**"
        if both.any() else "- No double-labeled grants yet.",
        "- Disagreements show how ambiguous 'relevant' is; they cap how well any method can score.", ""]
lines += ["## Labeled sample by stratum", "",
        df.groupby("stratum").agg(labeled=("relevant", "size"), relevant=("relevant", "sum"),
                                population=("population_size", "first")).astype(int).to_markdown(), ""]

jev_path = config.RESULTS / "jev_outputs.csv"
if jev_path.exists():
    jev = pd.read_csv(jev_path)[["grant_id", "route", "domain_relevance", "domain_relevance_p",
                                "applicant_role", "route_reason"]]
    df = df.merge(jev, on="grant_id", how="left")
    missing = df["route"].isna().sum()
    df["jev_retained"] = df["route"].fillna("deep_review").eq("deep_review")
    rates, ci = weighted_rates(df, "jev_retained"), bootstrap_ci(df, "jev_retained")
    n_retained = int(pd.read_csv(jev_path)["route"].eq("deep_review").sum())

    base = pd.read_csv(BASELINE)[["grant_id", "baseline_rank"]]
    df = df.merge(base, on="grant_id", how="left")
    df["baseline_retained"] = df["baseline_rank"].astype(float) <= n_retained
    brates, bci = weighted_rates(df, "baseline_retained"), bootstrap_ci(df, "baseline_retained")

    lines += ["## Layer 2 (Jev) vs keyword baseline", "",
            f"Jev sends **{n_retained}** grants to deep review. The baseline is given the same budget "
            f"(its top {n_retained} by TF-IDF similarity). Recall = share of relevant grants that reach Layer
3. "
            "Estimates are weighted by stratum size; intervals are 95% stratified bootstrap.", "",
            "| Method | Recall | 95% CI | Precision | 95% CI |", "|---|---|---|---|",
            f"| Jev routing | **{fmt(rates['recall'])}** | {fmt(ci['recall'][0])}-{fmt(ci['recall'][1])} | "
            f"| {fmt(rates['precision'])} | {fmt(ci['precision'][0])}-{fmt(ci['precision'][1])} |",
            f"| Keyword baseline (same budget) | {fmt(brates['recall'])} | {fmt(bci['recall'][0])}-"
            f"{fmt(bci['recall'][1])} | "
            f"| {fmt(brates['precision'])} | {fmt(bci['precision'][0])}-{fmt(bci['precision'][1])} |", ""]
    if missing:
        lines += [f"_{{missing}} labeled grants had no Jev output (API error?) and were counted as retained._", ""]
    fn = df[(df["relevant"] == 1) & ~df["jev_retained"]]
    lines += ["## Layer 2 false negatives (relevant but deprioritized) - inspect every one", ""]
    lines += [(f"- `{r.sample_id}` {r.title} - Jev: {r.domain_relevance} (p={r.domain_relevance_p}), "
            f"role {r.applicant_role}; {r.route_reason}" for r in fn.itertuples()) or ["- None in the
sample.")]
    lines += [""]
else:
    lines += ["## Layer 2", "", "_No jev_outputs.csv yet - run the pipeline, then re-run evaluate._", ""]

deep_path = config.RESULTS / "deep_analysis.csv"
if deep_path.exists():
    deep = pd.read_csv(deep_path)
    deep = deep[deep["status"] == "ok"][["grant_id", "recommended_for_review", "strategic_alignment"]]
    d = df.merge(deep, on="grant_id")
    if len(d):
        rec = d["recommended_for_review"].astype(str).str.lower().eq("true")

```

```

agree3 = (rec == (d["relevant"] == 1)).mean()
lines += ["## Layer 3 recommendation vs human label", "",
        f"- {len(d)} labeled grants were deep-analyzed; `recommended_for_review` matches the human
label "
        f"on {fmt(agree3)} (unweighted).",
        f"- Recommended but labeled not relevant: {int((rec & (d['relevant'] == 0)).sum())}; "
        f"labeled relevant but not recommended: {int((~rec & (d['relevant'] == 1)).sum())}.", ""]

err_path = config.RESULTS / "ai_error_log.csv"
if err_path.exists():
    err = pd.read_csv(err_path)
    err = err[err["check"] != "over_layer3_cap"]
    lines += ["## Where the AI was wrong (automatic checks, `ai_error_log.csv`)", ""]
    if len(err):
        lines += [err.groupby(["check",
"severity"]).size().rename("count").reset_index().to_markdown(index=False), "",
                "Examples:", ""]
        lines += [f"- **{r.check}** - {r.opportunity_title}: {r.detail}"
                for r in err[err["severity"] == "high"].head(5).itertuples()]
    else:
        lines += ["- No problems logged."]
    lines += [""]

config.RESULTS.mkdir(parents=True, exist_ok=True)
REPORT.write_text("\n".join(lines), encoding="utf-8")
print("\n".join(lines))
print(f"\nWrote {REPORT}")

if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd", required=True)
    s = sub.add_parser("sample")
    s.add_argument("--force", action="store_true")
    sub.add_parser("spotcheck")
    sub.add_parser("evaluate")
    a = ap.parse_args()
    {"sample": lambda: cmd_sample(a.force), "spotcheck": cmd_spotcheck, "evaluate": cmd_evaluate}[a.cmd]()

```

src/keyword_baseline.py

```
"""Layer 5 - keyword (TF-IDF) baseline. A comparison, not a pipeline feature.

Ranks every grant by lexical similarity to the agency's strategic plan
(mission, the 20 objectives, and the division scopes in priorities.json).
Each objective/division is its own document and a grant's score is its best
match, so a grant only has to fit one priority well.

validation.py compares this ranking against Jev's routing on the human-labeled
sample. If Jev does not beat it, the AI layer is not earning its place.

Output: data/results/keyword_baseline.csv
Usage: python src/keyword_baseline.py
"""

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import config

OUTPUT = config.RESULTS / "keyword_baseline.csv"

def plan_documents(agency: dict) -> dict[str, str]:
    docs = {}
    for p in agency["strategic_priorities"]:
        for o in p["objectives"]:
            docs[o["id"]] = f"{p['name']}. {p['description']} {o['text']}"
    for d in agency["divisions"]:
        docs[f"div:{d['id']}"] = f"{d['name']}. {d['scope']}"
    docs["mission"] = agency["mission"]
    return docs

def score(grants: pd.DataFrame, agency: dict) -> pd.DataFrame:
    docs = plan_documents(agency)
    texts = grants["grant_text"].fillna("").tolist()
    vec = TfidfVectorizer(stop_words="english", ngram_range=(1, 2), sublinear_tf=True, min_df=1)
    vec.fit(texts + list(docs.values()))
    sim = cosine_similarity(vec.transform(texts), vec.transform(list(docs.values())))
    keys = list(docs)
    out = grants[["grant_id", "opportunity_title", "hard_filtered"]].copy()
    out["baseline_score"] = sim.max(axis=1).round(5)
    out["baseline_best_match"] = [keys[i] for i in sim.argmax(axis=1)]
    # Rank only grants that reach Layer 2, so it is comparable with Jev's routing.
    live = ~out["hard_filtered"].astype(bool)
    out["baseline_rank"] = pd.NA
    out.loc[live, "baseline_rank"] = out.loc[live, "baseline_score"].rank(ascending=False,
method="first").astype(int)
    return out.sort_values("baseline_score", ascending=False)

def run() -> pd.DataFrame:
    grants = pd.read_csv(config.LAYER1_OUTPUT)
    out = score(grants, config.load_agency())
    config.RESULTS.mkdir(parents=True, exist_ok=True)
    out.to_csv(OUTPUT, index=False)
    print(f"Keyword baseline: scored {len(out)} grants -> {OUTPUT}")
    print(out[~out["hard_filtered"]].head(8)[["baseline_rank", "baseline_score", "opportunity_title"]
.to_string(index=False, max_colwidth=70))
    return out

if __name__ == "__main__":
    run()
```

src/report.py

```
"""Final deliverable: the top-results list for a non-technical reader.

Reads Layer 4's ranked_grants.csv and watchlist.csv and writes
data/results/top_results.md and top_results.csv - rank, grant, score, deadline,
award, owning division, matched goal, one-line rationale, main risk, and the
evidence status of the eligibility claim.

Usage:
python src/report.py [--top 10] [--watch 5] [--mock]
"""

import argparse

import pandas as pd

import config
import scoring

def _money(v) -> str:
    if pd.isna(v):
        return "not stated"
    return f"${v / 1e6:.1f}M" if v >= 1e6 else f"${v / 1e3:.0f}K"

def _truthy(v) -> bool:
    return str(v).strip().lower() == "true"

def build(ranked: pd.DataFrame, agency: dict, n: int, forecast: bool) -> pd.DataFrame:
    goals = {p["id"]: p["name"] for p in agency["strategic_priorities"]}
    divisions = {d["id"]: d["name"] for d in agency["divisions"]}
    live = ranked[ranked["score_status"] != "confirmed_ineligible"].head(n)
    rows = []
    for r in live.iteruples():
        if forecast:
            when = f"forecast close {r.forecasted_close_date}" if pd.notna(r.forecasted_close_date) else "forecast,
no date"
            if _truthy(getattr(r, "forecast_check_now", False)):
                when += " (forecast post date passed - may already be open, check Grants.gov now)"
            if _truthy(getattr(r, "forecast_close_passed", False)):
                when += " (date passed - check if posted)"
            elif pd.notna(r.close_date_real):
                when = f"{str(r.close_date_real)[:10]} ({int(r.days_until_close)} days)"
            else:
                if _truthy(getattr(r, "rolling_funds_limited", False)):
                    when = "rolling, first come first served - apply early, funds can run out"
                elif _truthy(getattr(r, "rolling_deadline", False)):
                    when = "rolling - accepted anytime"
                else:
                    when = "not stated"
            evidence = "verified quote" if _truthy(r.eligibility_quote_verified) else "UNVERIFIED - check eligibility"
            rows.append({
                "rank": r.rank, "grant": r.opportunity_title, "score": r.final_score, "deadline": when,
                "award (est.)": _money(r.award_value_usd),
                "owner": divisions.get(r.owning_division, r.owning_division),
                "goal": goals.get(r.matched_goal, r.matched_goal),
                "why": r.one_line_rationale, "main risk": r.major_risk,
                "DHHS role": f"{r.applicant_role_verified} ({evidence})",
                "cost share": r.requires_cost_share, "link": r.url,
            })
    return pd.DataFrame(rows)

def run(top: int = 10, watch: int = 5, mock: bool = False) -> None:
    agency = config.load_agency()
```

```

out_dir = config.MOCK_RESULTS if mock else config.RESULTS
open_list = build(pd.read_csv(out_dir / "ranked_grants.csv"), agency, top, forecast=False)
watch_list = build(pd.read_csv(out_dir / "watchlist.csv"), agency, watch, forecast=True)
pd.concat([open_list.assign(list="apply now"), watch_list.assign(list="watchlist")]).to_csv(
    out_dir / "top_results.csv", index=False)

weights, *_ = scoring.scoring_config(agency)
formula = ", ".join(f"{k.replace('_', '').replace(' ', ' ')} {v:.0%}" for k, v in weights.items())
md = [f"## Top federal grant opportunities for {agency['agency_short']}",
      f"_As of {agency['as_of_date']}. Scores are 0-100 from a fixed, published formula ({formula})._",
      "" if not mock else "\n**MOCK RUN - fake model answers, do not use.**\n",
      f"## Apply now: top {len(open_list)} open opportunities", ""]
for r in open_list.to_dict("records"):
    md += [f"### {r['rank']}. {r['grant']} - {r['score']:.0f}/100",
          f"- **Why:** {r['why']}",
          f"- **Owner:** {r['owner']} · **Goal:** {r['goal']} · **DHHS role:** {r['DHHS role']}",
          f"- **Deadline:** {r['deadline']} · **Award:** {r['award (est.)']} · **Cost share:** {r['cost share']}",
          f"- **Main risk:** {r['main risk']}", f"- [Grant listing]({r['link']})", ""]
md += [f"## Watchlist: {len(watch_list)} forecasted opportunities to prepare for", ""]
for r in watch_list.to_dict("records"):
    md += [f"- **{r['grant']}** ({r['score']:.0f}/100) - {r['why']} Owner: {r['owner']}. "
          f"{r['deadline']}. [Listing]({r['link']})"]
research_path = out_dir / "research_partnerships.csv"
if research_path.exists():
    research = build(pd.read_csv(research_path), agency, 5, forecast=False)
    md += ["", "## Research partnerships: forward to a university partner",
          "_NIH-style research and training awards (R01, R34, K99 ...). DHHS can rarely lead these, "
          "but a partner such as UNC may want DHHS data or collaboration. Listed separately so they "
          "do not crowd out program funding._", ""]
    for r in research.to_dict("records"):
        md += [f"- **{r['grant']}** ({r['score']:.0f}/100) - {r['why']} Owner: {r['owner']}. [Listing]({r['link']})"]
(out_dir / "top_results.md").write_text("\n".join(md) + "\n", encoding="utf-8")
print(f"Wrote {out_dir / 'top_results.md'} and top_results.csv")

if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--top", type=int, default=10)
    ap.add_argument("--watch", type=int, default=5)
    ap.add_argument("--mock", action="store_true")
    a = ap.parse_args()
    run(a.top, a.watch, a.mock)

```

src/submission_table.py

```
"""Submission item 1: the top-results table, one row per grant, rationale tied to our criteria.

Writes submission/top_results.csv (opens in any spreadsheet) and submission/top_results.md.
Rows: the top 10 open grants and the top 5 watchlist grants. Each row has the model's
one-line rationale plus a criteria check built deterministically from the five criteria
(agency/agency_profile.md), and the USAspending award history for NC DHHS.

Usage: python src/submission_table.py
"""

import pandas as pd

import config

OUT = config.ROOT / "submission"
ROLE_TEXT = {"lead": "DHHS can lead", "partner": "DHHS as partner", "atypical": "eligible but atypical",
             "unclear": "eligibility unclear", "ineligible": "ineligible"}

def _money(v) -> str:
    if pd.isna(v) or v <= 0:
        return "not stated"
    return f"${v / 1e6:.1f}M" if v >= 1e6 else f"${v / 1e3:.0f}K"

def _true(v) -> bool:
    return str(v).strip().lower() == "true"

def rows(ranked: pd.DataFrame, hist: pd.DataFrame, agency: dict, n: int, track: str) -> list[dict]:
    goals = {p["id"]: p["name"] for p in agency["strategic_priorities"]}
    divs = {d["id"]: d["name"] for d in agency["divisions"]}
    nc_min = agency["layer5_award_history"]["nc_dhhs_min_usd"]
    h = hist.set_index("grant_id")
    out = []
    for r in ranked[ranked["score_status"] != "confirmed_ineligible"].head(n).itertuples():
        if track == "watchlist":
            deadline = f"forecast close {r.forecasted_close_date}" if pd.notna(r.forecasted_close_date) else
"forecast, no date"
            if _true(getattr(r, "forecast_check_now", False)):
                deadline += " - may already be open, check Grants.gov now"
            elif _true(getattr(r, "forecast_close_passed", False)):
                deadline += " - date passed, check if posted"
            elif pd.notna(r.close_date_real):
                deadline = f"{str(r.close_date_real)[:10]} ({int(r.days_until_close)} days)"
            else:
                deadline = "rolling" if _true(getattr(r, "rolling_deadline", False)) else "not stated"
            role = r.applicant_role_verified
            evidence = "quote verified" if _true(r.eligibility_quote_verified) else "quote NOT verified"
            criteria = (f"1 Eligibility: {ROLE_TEXT.get(role, role)} ({evidence}) | "
                        f"2 Goal: {goals.get(r.matched_goal, 'none matched')} ({r.strategic_alignment:.0f}/10) | "
                        f"3 Owner: {divs.get(r.owning_division, r.owning_division)} (fit {r.operational_fit:.0f}/10) | "
                        f"4 Timing: {deadline} | 5 Award: {_money(r.award_value_usd)}, cost share: "
                        f"{ {'True': 'yes', 'False': 'no'}.get(str(r.requires_cost_share), 'unknown') }")
            nc = h.loc[r.grant_id, "nc_dhhs"] if r.grant_id in h.index else 0.0
            out.append({
                "list": "Apply now" if track == "open" else "Prepare now (forecast)",
                "rank": int(r.rank), "grant": r.opportunity_title, "score_0_100": round(float(r.final_score)),
                "one_line_rationale": r.one_line_rationale, "criteria_check": criteria,
                "main_risk": r.major_risk,
                "nc_dhhs_awards_fy22_25": _money(nc) if nc >= nc_min else "none on record",
                "past_recipients": h.loc[r.grant_id, "verdict"] if r.grant_id in h.index else "",
                "link": r.url,
            })
    return out
```

```

def run() -> pd.DataFrame:
    agency = config.load_agency()
    hist = pd.read_csv(config.RESULTS / "award_history.csv")
    table = pd.DataFrame(rows(pd.read_csv(config.RESULTS / "ranked_grants.csv"), hist, agency, 10, "open")
        + rows(pd.read_csv(config.RESULTS / "watchlist.csv"), hist, agency, 5, "watchlist"))
    OUT.mkdir(exist_ok=True)
    table.to_csv(OUT / "top_results.csv", index=False, encoding="utf-8-sig") # BOM: Excel shows dashes right

    md = [f"# Top federal grant opportunities for {agency['agency_name']}", "",
        f"As of {agency['as_of_date']} (data pulled 2026-08-18). Score: 0-100 from a published formula "
        "(strategic fit 30%, ability to run it 25%, award size 15%, deadline 15%, eligibility 15%). "
        "NC DHHS award history: USAspending.gov, FY2022-FY2025. The criteria check numbers match our five "
        "criteria in `agency/agency_profile.md`. ", ""]
    for name in ["Apply now", "Prepare now (forecast)"]:
        md += [f"## {name}", "", "| # | Grant | Score | Why (one line) | NC DHHS won FY22-25 |", "|---|---|---|---|--"
            -"|"]
        for r in table[table["list"] == name].itertuples():
            md.append(f"| {r.rank} | [{r.grant}][({r.link})] | {r.score_0_100} | {r.one_line_rationale} |"
                f"{r.nc_dhhs_awards_fy22_25} |")
        md += [""]
        md += ["## Criteria check and main risk, per grant", ""]
        for r in table.itertuples():
            md += [f"*{r.list}#{r.rank}: {r.grant}*", f"- Criteria: {r.criteria_check}",
                f"- Main risk: {r.main_risk}", f"- Past recipients under this program: {r.past_recipients}", ""]
        (OUT / "top_results.md").write_text("\n".join(md), encoding="utf-8")
    print(f"Wrote {OUT / 'top_results.csv'} and {OUT / 'top_results.md'} ({len(table)} grants)")
    return table

if __name__ == "__main__":
    run()

```

src/clients.py

```
"""API clients for Jev (Layer 2) and the deep-analysis chat model (Layer 3).

Both go through the Vercel AI Gateway with one key (AI_GATEWAY_API_KEY in .env).
The Mock* clients let anyone dry-run the pipeline without a key; their outputs are
fake and are written to separate folders (data/cache_mock, data/results/mock).
"""

import json
import re
import time

import requests

import config

class APIError(RuntimeError):
    pass

def _post(url: str, body: dict, retries: int = 4) -> dict:
    key = config.api_key()
    if not key:
        raise APIError("AI_GATEWAY_API_KEY is not set (copy .env.example to .env), "
                        "or run with --mock for a dry run.")
    headers = {"Authorization": f"Bearer {key}"}
    for attempt in range(retries):
        r = requests.post(url, headers=headers, json=body, timeout=config.REQUEST_TIMEOUT_S)
        if r.status_code == 200:
            return r.json()
        if r.status_code in (429, 500, 502, 503, 504) and attempt < retries - 1:
            time.sleep(2 ** attempt)
            continue
        raise APIError(f"HTTP {r.status_code} from {url}: {r.text[:200]}")
    raise APIError("unreachable")

class JevClient:
    """Jev answers typed `choice` questions about a `state` and returns, for each
    question, the chosen option plus a probability for every option."""

    model = config.JEV_MODEL

    def evaluate(self, state: dict, questions: dict) -> dict:
        data = _post(config.GATEWAY_EVALUATE_URL,
                     {"model": self.model, "state": state, "questions": questions})
        return data.get("answers") or data.get("results") or {}

class ChatClient:
    """OpenAI-compatible chat completions through the gateway."""

    model = config.DEEP_MODEL

    def complete(self, messages: list[dict]) -> str:
        data = _post(config.GATEWAY_CHAT_URL,
                     {"model": self.model, "messages": messages, "temperature": 0})
        return data["choices"][0]["message"]["content"]

# — Mock clients (dry runs only; outputs are NOT real model judgments) —

class MockJevClient:
    model = "mock-jev"

    def evaluate(self, state: dict, questions: dict) -> dict:
```

```

text = " ".join(str(v) for v in state.get("grant", {}).values()).lower()
health = len(re.findall(r"health|medicaid|behavioral|substance|child|maternal|aging|disabilit", text))
answers = {}
for name, q in questions.items():
    options = list(q["criteria"])
    # Put most mass on one option picked from a simple keyword count.
    if name == "domain_relevance":
        pick = options[min(health, len(options) - 1)]
    else:
        pick = options[(len(text) + len(name)) % len(options)]
    rest = (1 - 0.7) / max(len(options) - 1, 1)
    answers[name] = {"choice": pick,
                    "probabilities": {o: (0.7 if o == pick else rest) for o in options}}
return answers

```

```

class MockChatClient:
    model = "mock-chat"

    def complete(self, messages: list[dict]) -> str:
        user = messages[-1]["content"]
        m = re.search(r"ELIGIBILITY TEXT:\n(.{20,80}?)[\n]", user)
        quote = m.group(1).strip() if m else ""
        return json.dumps({
            "strategic_alignment": 5, "matched_goal": "G1", "matched_objective": "G1.03",
            "strategic_reason": "MOCK output - not a real judgment.",
            "strategic_evidence_quote": quote,
            "applicant_role": "unclear", "eligibility_evidence_quote": quote,
            "owning_division": "OSEC",
            "operational_fit": 5, "operational_reason": "MOCK",
            "restrictions": [], "major_risk": "MOCK",
            "recommended_for_review": False,
            "one_line_rationale": "MOCK output - run without --mock for real results.",
        })

```

src/llm_cache.py

```
"""On-disk cache of every model response.

Each response is stored as one JSON file keyed by a hash of (model, prompt version,
exact request). The cache is committed to the repo, so `python src/pipeline.py` can
reproduce the ranking with no API key and no cost. Changing a prompt changes the
hash, so stale answers are never reused by accident.
"""

import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path

def request_hash(model: str, prompt_version: str, request) -> str:
    blob = json.dumps({"model": model, "prompt_version": prompt_version, "request": request},
                      sort_keys=True, ensure_ascii=False)
    return hashlib.sha256(blob.encode("utf-8")).hexdigest()

def prompt_version(*texts: str) -> str:
    """Short hash identifying a prompt template (recorded with every cached answer)."""
    return hashlib.sha256("\n".join(texts).encode("utf-8")).hexdigest()[:12]

class ResponseCache:
    def __init__(self, directory: Path, layer: str):
        self.dir = Path(directory) / layer
        self.dir.mkdir(parents=True, exist_ok=True)

    def _path(self, grant_id: str, key: str) -> Path:
        safe = "".join(c for c in str(grant_id) if c.isalnum() or c in "-_")[:80]
        return self.dir / f"{safe}_{key[:16]}.json"

    def get(self, grant_id: str, key: str):
        p = self._path(grant_id, key)
        if p.exists():
            with open(p, encoding="utf-8") as f:
                return json.load(f)["response"]
        return None

    def put(self, grant_id: str, key: str, model: str, version: str, response) -> None:
        record = {
            "grant_id": grant_id,
            "model": model,
            "prompt_version": version,
            "request_hash": key,
            "retrieved_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
            "response": response,
        }
        with open(self._path(grant_id, key), "w", encoding="utf-8") as f:
            json.dump(record, f, ensure_ascii=False, indent=1)
```

src/text_utils.py

```
"""Text cleanup and the verbatim-quote check used to catch hallucinated evidence."""

import html
import re

_TAG = re.compile(r"<[^>+>")
_WS = re.compile(r"\s+")
_PUNCT_MAP = str.maketrans({
    "€": "'", "": "'", "": "'", "€": "'", "": "'",
    "-": "-", "-": "-", " ": " ",
})

def strip_html(text) -> str:
    """Grant summaries often contain raw HTML (<p>, <br>, &amp;). Return plain text."""
    if not isinstance(text, str):
        return ""
    text = re.sub(r"(?i)<br\s*/?>|</p>|</li>", "\n", text)
    text = _TAG.sub(" ", text)
    return html.unescape(text).strip()

def normalize(text: str) -> str:
    """Lowercase, unify quotes/dashes, collapse whitespace. Used only for matching."""
    text = strip_html(text).translate(_PUNCT_MAP).lower()
    return _WS.sub(" ", text).strip()

def quote_in_source(quote, source: str, min_chars: int = 12) -> bool:
    """True if `quote` appears verbatim (after normalization) in `source`.

    Models sometimes shorten a quote with "..." -- each fragment must then appear
    in order. Quotes shorter than `min_chars` are rejected: too short to be evidence.
    """
    if not isinstance(quote, str) or not quote.strip():
        return False
    src = normalize(source)
    fragments = [normalize(f) for f in re.split(r"\.\\.\\.|...", quote)]
    fragments = [f.strip(" \'"') for f in fragments if f.strip(" \'"')]
    if not fragments or sum(len(f) for f in fragments) < min_chars:
        return False
    pos = 0
    for frag in fragments:
        i = src.find(frag, pos)
        if i < 0:
            return False
        pos = i + len(frag)
    return True

def truncate(text: str, max_chars: int) -> str:
    return text if len(text) <= max_chars else text[:max_chars].rsplit(" ", 1)[0] + " [truncated]"
```

prompts/jev_triage.json

```
{
  "_comment": "Layer 2 question set for Jev. Every question is a `choice` question, so Jev returns a probability for every option; those probabilities are our confidence signal. Options marked FROM_CONFIG are filled from agency/priorities.json. Editing this file changes the prompt hash, so cached answers are not reused.",
  "context": "You are triaging federal grant opportunities for {agency_name}. Mission: {mission} The person using these results is {user} A grant matters only if this agency itself could realistically apply for it or be a named partner on it.",
  "questions": {
    "domain_relevance": {
      "type": "choice",
      "instructions": "How closely is this grant's subject matter tied to the agency's mission and strategic goals? Judge the actual funded activity, not shared buzzwords. Basic laboratory or clinical research aimed at universities is at most 'weak' unless it funds state public-health, Medicaid, or human-services programs.",
      "criteria": {
        "none": "Unrelated to health or human services in North Carolina (for example defense research, foreign-embassy programs, space science, arts).",
        "weak": "Loosely health-related, but the funded work is something the agency does not do (for example basic biomedical research, drug discovery, overseas projects).",
        "moderate": "Funds work in the agency's field that the agency or its divisions could plausibly run, but it matches no strategic goal directly.",
        "strong": "Funds programs, services, infrastructure or workforce that directly advance one of the agency's strategic goals."
      }
    },
    "matched_goal": {
      "type": "choice",
      "instructions": "Which single strategic goal would this grant advance most? Pick 'none' if it advances none of them.",
      "criteria": "FROM_CONFIG:strategic_priorities"
    },
    "applicant_role": {
      "type": "choice",
      "instructions": "Read the eligibility text and applicant types. What role could a state health and human services agency realistically play? 'State governments' appearing in a long list of eligible applicant types does NOT make the state the intended applicant: research grants often list every applicant type.",
      "criteria": {
        "lead": "A state agency or state health department is an intended lead applicant (for example the text names states, state health departments, state Medicaid agencies, or state-administered formula programs).",
        "partner": "The lead is usually someone else (university, county, nonprofit, tribe), but a state agency is a natural required or expected partner, subrecipient, or data provider.",
        "atypical": "A state agency is technically eligible but the program is clearly designed for other applicants such as individual researchers, universities, or small businesses.",
        "ineligible": "The eligibility text excludes state agencies, or limits applicants to categories the agency is not (for example only tribes, only foreign organizations, only individuals).",
        "unclear": "The eligibility information is missing or too vague to decide."
      }
    },
    "owning_division": {
      "type": "choice",
      "instructions": "If the agency pursued this grant, which division or office would most likely own the application? Pick 'none' if no division fits.",
      "criteria": "FROM_CONFIG:divisions"
    }
  }
}
```

prompts/deep_analysis.txt

```
=== SYSTEM ===
You are a careful grants analyst for {agency_name}. You extract evidence from federal grant announcements. You never invent facts. Every quote you give must be copied character-for-character from the grant text supplied by the user; if no supporting text exists, return an empty string for that quote. A program will check every quote against the source text, and any quote that is not found verbatim will be flagged as an error.

You do NOT decide the final ranking and you do NOT judge deadlines or award size; those are computed separately in Python. Answer only with one JSON object, no prose before or after it.

=== USER ===
AGENCY: {agency_name}
MISSION: {mission}
USER OF THESE RESULTS: {user}

STRATEGIC GOALS AND OBJECTIVES (use these ids):
{priorities_block}

DIVISIONS (use these ids):
{divisions_block}

GRANT
Title: {title}
Federal agency: {federal_agency}
Assistance listings: {assistance_listings}
Funding instrument: {funding_instruments}
Structured applicant types: {applicant_types}

SUMMARY TEXT:
{summary}

ELIGIBILITY TEXT:
{eligibility}

Return this JSON object:
{{
  "strategic_alignment": <integer 0-10: how directly the funded activity advances one of the agency's objectives; 0 = unrelated, 10 = directly funds a named strategy>,
  "matched_goal": "<goal id such as G3, or none>",
  "matched_objective": "<objective id such as G3.04, or none>",
  "strategic_reason": "<one sentence linking the funded activity to that objective>",
  "strategic_evidence_quote": "<verbatim sentence or phrase from the SUMMARY TEXT or title that shows what the grant funds>",
  "applicant_role": "<lead | partner | atypical | ineligible | unclear>",
  "eligibility_evidence_quote": "<verbatim phrase from the ELIGIBILITY TEXT, SUMMARY TEXT or applicant types that shows whether a state agency can apply; empty string if there is none>",
  "owning_division": "<division id, or none>",
  "operational_fit": <integer 0-10: could this division realistically run the program with its existing role, authority and partners; 0 = outside its authority, 10 = an extension of work it already does>,
  "operational_reason": "<one sentence>",
  "restrictions": ["<each condition that adds cost or effort: cost sharing, required partners, geographic limits, unusual reporting; empty list if none stated>"],
  "major_risk": "<the single biggest reason this grant could be less valuable than it looks>",
  "recommended_for_review": <true if the grants coordinator should spend time on this, otherwise false>,
  "one_line_rationale": "<at most 30 words for a non-technical budget director: why this grant does or does not fit, tied to the goal and the agency's role>"
}}
```

agency/priorities.json

```
{
  "agency_name": "North Carolina Department of Health and Human Services (NCDHHS)",
  "agency_short": "NCDHHS",
  "source": "NCDHHS Strategic Plan 2023-2025, https://www.osbm.nc.gov/strategic-plan-ncdhhs/open",
  "mission": "In collaboration with our partners, the North Carolina Department of Health and Human Services provides essential services to improve the health, safety, and well-being of all North Carolinians.",
  "user": "The NCDHHS federal-grants coordinator who reviews new Grants.gov postings each week, decides which are worth pursuing, and routes each one to the division that would own the application.",
  "as_of_date": "2026-09-30",
  "strategic_priorities": [
    {
      "id": "G1",
      "name": "Health Access",
      "description": "Advance health access by increasing opportunity and outcomes for people who face greater health and situational challenges.",
      "objectives": [
        {
          "id": "G1.01",
          "text": "Build a culture and structure for increasing health access and reducing health disparities across the Department."
        },
        {
          "id": "G1.02",
          "text": "Use data, metrics and dashboards to track health disparities and outcomes and make benefits easier to apply for."
        },
        {
          "id": "G1.03",
          "text": "Mitigate barriers to health services: telehealth and broadband, language access, culturally competent communication, rural and underserved primary care."
        },
        {
          "id": "G1.04",
          "text": "Invest in partner capacity: community health workers, community-based organizations, justice-involved people's health access, Medicaid expansion enrollment."
        }
      ]
    },
    {
      "id": "G2",
      "name": "Child and Family Well-Being",
      "description": "Promote child and family well-being by making it easier for children and families to access the healthcare, programs, and supports they need.",
      "objectives": [
        {
          "id": "G2.01",
          "text": "Meet the behavioral and mental health needs of children and families (youth mental health first aid, school health records, NC-PAL, specialty care for children with complex needs)."
        },
        {
          "id": "G2.02",
          "text": "Reform child welfare and social services to improve safety, permanency and well-being (foster care, child welfare information systems, county support)."
        },
        {
          "id": "G2.03",
          "text": "Increase food security for children and families (WIC, FNS/SNAP enrollment, NCCARE360 referrals, breastfeeding support, Healthy Opportunities pilots)."
        },
        {
          "id": "G2.04",
          "text": "Improve women's health and birth outcomes (prenatal care, maternal and newborn levels of care, doulas, reproductive life planning, infant mortality)."
        }
      ]
    }
  ]
}
```

```

{
  "id": "G3",
  "name": "Behavioral Health and Resilience",
  "description": "Support behavioral health and resilience through coordinated systems of care that make mental health and substance use services easy to access and reduce stigma.",
  "objectives": [
    {
      "id": "G3.01",
      "text": "Integrate behavioral and physical health care (Collaborative Care, certified community behavioral health clinics, Tailored Care Management).",
    },
    {
      "id": "G3.02",
      "text": "Expand behavioral health supports for people involved in the justice system (capacity restoration, diversion, re-entry).",
    },
    {
      "id": "G3.03",
      "text": "Increase access to crisis services (988, suicide prevention, statewide behavioral health referral network).",
    },
    {
      "id": "G3.04",
      "text": "Turn the tide on the opioid and substance use crisis (evidence-based treatment, harm reduction, drug checking, collegiate recovery).",
    },
    {
      "id": "G3.05",
      "text": "Improve behavioral health data infrastructure (behavioral health holds data, syndromic surveillance, electronic health records in state psychiatric hospitals).",
    }
  ],
},
{
  "id": "G4",
  "name": "Strong and Inclusive Workforce",
  "description": "Build a strong and inclusive workforce that supports early learning, health, and wellness across North Carolina.",
  "objectives": [
    {
      "id": "G4.01",
      "text": "Invest in the caregiving and direct care workforce serving older adults and people with disabilities.",
    },
    {
      "id": "G4.02",
      "text": "Invest in the early childhood workforce (teacher credentialing, educator compensation).",
    },
    {
      "id": "G4.03",
      "text": "Make NCDHHS an equitable workplace (recruitment, training, retention of Department staff).",
    },
    {
      "id": "G4.04",
      "text": "Support the public health and child welfare workforce at state and local levels.",
    }
  ],
},
{
  "id": "G5",
  "name": "Operational Excellence",
  "description": "Achieve operational excellence by enabling efficient, effective, and innovative processes and services.",
  "objectives": [
    {
      "id": "G5.01",
      "text": "Foster data-driven decision-making (data infrastructure, integration, dashboards, staff data skills).",
    },
  ],
}

```

```

        "id": "G5.02",
        "text": "Be effective stewards of state resources (tracking federal funding and block grants, contracting,
websites, energy, invoicing).",
    },
    {
        "id": "G5.03",
        "text": "Work across departments through shared processes and collaboration."
    }
]
},
"divisions": [
    {
        "id": "DPH",
        "name": "Division of Public Health",
        "scope": "disease prevention and surveillance, epidemiology, immunization, chronic disease, environmental
health, maternal and child health, WIC, local health departments, public health preparedness, laboratory"
    },
    {
        "id": "DMHDDSUS",
        "name": "Division of Mental Health, Developmental Disabilities and Substance Use Services",
        "scope": "mental health, substance use and opioid services, intellectual and developmental disabilities, crisis
services and 988, behavioral health for justice-involved people"
    },
    {
        "id": "MEDICAID",
        "name": "NC Medicaid (Division of Health Benefits)",
        "scope": "Medicaid and CHIP coverage, managed care, Medicaid expansion, Healthy Opportunities pilots, provider
reimbursement"
    },
    {
        "id": "DCFW",
        "name": "Division of Child and Family Well-Being",
        "scope": "child behavioral health, school health, food and nutrition programs for children and families, home
visiting, early intervention"
    },
    {
        "id": "DSS",
        "name": "Division of Social Services",
        "scope": "child welfare, foster care, adoption, adult protective services, FNS/SNAP administration, TANF/Work
First, child support"
    },
    {
        "id": "DAGING",
        "name": "Division of Aging",
        "scope": "older adults, family caregivers, adult services, Older Americans Act programs"
    },
    {
        "id": "DCDEE",
        "name": "Division of Child Development and Early Education",
        "scope": "child care, early learning, NC Pre-K, child care subsidy, early childhood workforce"
    },
    {
        "id": "ORH",
        "name": "Office of Rural Health",
        "scope": "rural and underserved primary care, safety-net clinics, rural hospitals, health workforce shortage
areas, farmworker health"
    },
    {
        "id": "DEIPD",
        "name": "Division of Employment and Independence for People with Disabilities",
        "scope": "vocational rehabilitation, independent living, employment for people with disabilities"
    },
    {
        "id": "DSBDHH",
        "name": "Divisions of Services for the Blind / Deaf and Hard of Hearing",
        "scope": "services and accessibility for people who are blind, deaf, or hard of hearing"
    },
    {
        "id": "DHSR",

```

```

    "name": "Division of Health Service Regulation",
    "scope": "licensing and certification of health facilities, emergency medical services, certificate of need"
  },
  {
    "id": "DSOHF",
    "name": "Division of State Operated Healthcare Facilities",
    "scope": "state psychiatric hospitals, developmental centers, alcohol and drug treatment centers"
  },
  {
    "id": "OSEC",
    "name": "Office of the Secretary / central administration",
    "scope": "department-wide data, IT, HR, finance, and cross-division initiatives"
  }
],
"layer1": {
  "_comment": "Objective thresholds for Layer 1 (src/deterministic.py). Nothing here is semantic.
deadline_score_points are [days_until_close, score 0-10] anchors, linearly interpolated; past the last anchor the
score stays at its value. financial_value_score is log-scaled between the two USD anchors; total_only_cap limits
grants where only total program funding (not a per-award figure) is known.",
  "data_pulled_date": "2026-08-18",
  "close_date_placeholder_year_min": 2090,
  "deadline_days": [
    7,
    14,
    30
  ],
  "deadline_score_points": [
    [
      0,
      0
    ],
    [
      7,
      2
    ],
    [
      14,
      4
    ],
    [
      30,
      7
    ],
    [
      60,
      10
    ]
  ],
  "large_award_usd": 1000000,
  "few_awards_max": 3,
  "min_description_chars": 100,
  "broad_applicant_types_min": 12,
  "financial_log_min_usd": 100000,
  "financial_log_max_usd": 10000000,
  "financial_total_only_cap": 7.0,
  "financial_unknown_score": 5.0
},
"layer2_routing": {
  "_comment": "Probability cutoffs on Jev's per-option probabilities. A grant is deprioritized only when Jev is
confident it is irrelevant or not ours; everything else goes to Layer 3. Defaults are deliberately conservative
(recall over precision) and must be re-tuned on the labeled validation sample.",
  "deprioritize_if_p_none_at_least": 0.85,
  "deprioritize_if_p_none_or_weak_at_least": 0.95,
  "deprioritize_if_p_ineligible_at_least": 0.9
},
"layer3": {
  "max_grants": null,
  "_comment": "null = deep-analyze every grant Layer 2 routes to deep review. Set an integer to cap cost; overflow
is logged, never silently dropped."
},

```

```

"layer4_scoring": {
  "_comment": "Deterministic final ranking. Weights sum to 1.0. Forecasts omit deadline and re-normalize without
penalty. Open grants with a missing deadline re-normalize, then receive a small uncertainty penalty on the 100-point
final scale. Rolling ('accepted anytime') grants get a smaller penalty than a truly unknown deadline; rolling grants
whose funds can run out (first come, first served) get none.",
  "weights": {
    "strategic_alignment": 0.3,
    "operational_fit": 0.25,
    "financial_value_score": 0.15,
    "deadline_score": 0.15,
    "eligibility_confidence_score": 0.15
  },
  "eligibility_confidence_by_verified_role": {
    "lead": 10,
    "partner": 10,
    "atypical": 10,
    "unclear": 5,
    "ineligible": 0
  },
  "missing_open_deadline_penalty_points": 3,
  "rolling_deadline_penalty_points": 2,
  "rolling_funds_limited_penalty_points": 0,
  "unverified_strategic_evidence_penalty_points": 5
},
"layer5_award_history": {
  "_comment": "Objective, non-AI check using USAspending.gov: who actually received grant awards (block, formula,
project grants, cooperative agreements) under each grant's assistance listings in recent fiscal years. Answers 'is
this a program state agencies like DHHS actually win', not strategic fit. Recipient classes come from name rules in
src/award_history.py because USAspending's own recipient-type tags are incomplete (e.g. ALN 93.917: NC DHHS received
$187.6M by name, but only $91.2M is tagged as state government). NC DHHS counts as a past recipient only above
nc_dhhs_min_usd in total, so a tiny award under a broad shared listing does not count as evidence.",
  "start_date": "2021-10-01",
  "end_date": "2025-09-30",
  "fiscal_years_label": "FY2022-FY2025",
  "award_type_codes": [
    "02",
    "03",
    "04",
    "05"
  ],
  "national_recipient_pages": 2,
  "state_agency_share_threshold": 0.25,
  "majority_share": 0.5,
  "min_classified_coverage": 0.5,
  "nc_dhhs_min_usd": 100000
}
}

```